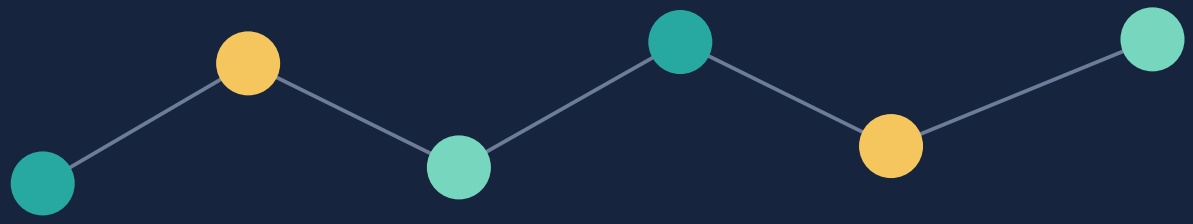
GATE 2027 / COMPLETE SYLLABUS EDITION

Computer Organization Architecture

Start from the basics. Build every syllabus idea. Then practise.

Section 3: Computer Organization and Architecture



LEARN → SEE → WORK THROUGH → RETRIEVE → PRACTISE

1 Understand

Each chapter starts in plain language.

2 Visualize

Follow the flow and redraw it.

3 Work

Trace the worked example with pencil.

4 Recall

Answer before checking the solution.

A beginner learning guide + illustrated recall/practice + your supplied reference material

2027 syllabus coverage map

Use this as a route map: every item below has a beginner lesson in this PDF. The supplied handbook/chapter PDFs are gathered in the reference section after the learning and practice pages.

- [ ] **Instructions and addressing modes**

  Instruction fields • Addressing mode • Effective address • Instruction cycle

- [ ] **ALU and arithmetic**

  Adder building blocks • Subtraction • Flags • Shifts

- [ ] **Control unit: hardwired and microprogrammed**

  Hardwired control • Microprogrammed control • Micro-operations • Compare carefully

- [ ] **Memory hierarchy and performance**

  Hierarchy • Locality • Performance • Memory interface

- [ ] **Cache mapping**

  Direct mapped • Set associative • Address fields • Replacement and writes

- [ ] **Interrupts and DMA**

  Polling • Interrupt • DMA • Trade-off

- [ ] **Instruction pipelining and hazards**

  Stages • Structural hazard • Data hazard • Control hazard

- [ ] **Cache address fields and multi-level AMAT**

  Address split • Direct and set associative • Two-level AMAT • Write policies

- [ ] **IEEE 754 numbers, hazards and pipeline time**

  Special IEEE values • Precision and rounding • Data hazards • Pipeline timing

Your route through this subject

Follow the lessons in order. Tick a topic only after you can explain its picture without looking; a tick means “I can teach it,” not “I have seen it.”

1. Instructions and addressing modes
2. ALU and arithmetic
3. Control unit: hardwired and microprogrammed
4. Memory hierarchy and performance
5. Cache mapping
6. Interrupts and DMA
7. Instruction pipelining and hazards
8. Cache address fields and multi-level AMAT
9. IEEE 754 numbers, hazards and pipeline time

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

IF THIS SUBJECT FEELS COMPLETELY NEW

That is okay. Start with the first lesson, keep a pencil beside you, and redraw its picture once. Attempt the recall prompt before opening the answer guide.

Instructions and addressing modes

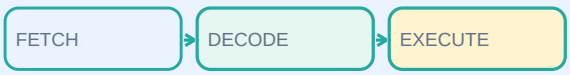
START WITH THE IDEA

A program is a sequence of encoded instructions. The instruction set defines available operations and how operands are specified.

A CONCRETE FIRST EXAMPLE

For LOAD R1, 8(R2), effective address=contents(R2)+8; memory at that address is loaded into R1.

PICTURE IT



IMAGINE IT THIS WAY

The opcode is the task; addressing mode is the route to its ingredients.

THE IDEAS, ONE BY ONE

Instruction fields

Opcode identifies the operation; register, immediate or address fields identify inputs and destination.

Addressing mode

Immediate means the value is inside the instruction; register means use a register; direct/indirect use memory addresses; indexed adds a register offset.

Effective address

Calculate the address explicitly from the given mode before fetching data.

Instruction cycle

A simplified flow is fetch, decode, operand read, execute, memory access if needed, write back.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Keep address, data stored there and address of an address distinct.

TRY FROM MEMORY

Cover this page. Explain Instructions and addressing modes using the picture, then give one example before looking again.

ALU and arithmetic

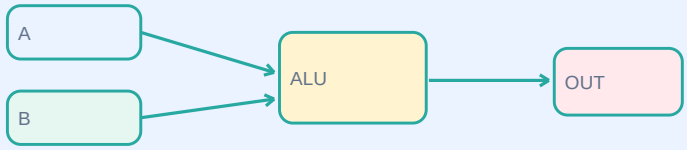
START WITH THE IDEA

The arithmetic and logic unit performs the data operations selected by an instruction: addition, subtraction, logic, shifts and comparisons.

A CONCRETE FIRST EXAMPLE

Four-bit signed 7+3: 0111+0011=1010. Positive + positive produced a negative-looking result, so signed overflow occurred.

PICTURE IT



IMAGINE IT THIS WAY

The ALU is a multi-tool; control lines select which operation is active.

THE IDEAS, ONE BY ONE

Adder building blocks

Half/full adders combine bit sums and carries; carry propagation can dominate delay.

Subtraction

Two's complement converts subtraction into addition of the complemented second operand plus one.

Flags

Zero, sign, carry and overflow flags describe different properties of a result.

Shifts

Logical shifts insert zeros; arithmetic right shift preserves a signed sign bit, subject to the machine definition.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Carry-out does not by itself signal signed overflow.

TRY FROM MEMORY

Cover this page. Explain ALU and arithmetic using the picture, then give one example before looking again.

Control unit: hardwired and microprogrammed

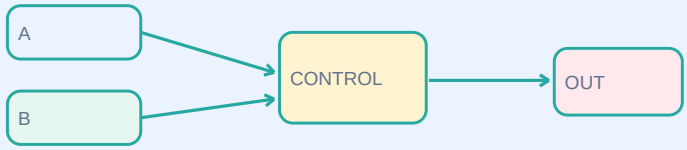
START WITH THE IDEA

The control unit coordinates register transfers, ALU operations, memory reads and writes over time. It turns an instruction into a sequence of control signals.

A CONCRETE FIRST EXAMPLE

During fetch, PC identifies the next instruction address; after memory returns the instruction, it is stored in IR for decode.

PICTURE IT



IMAGINE IT THIS WAY

Hardwired control is a fixed mechanism; microcode is a tiny internal recipe.

THE IDEAS, ONE BY ONE

Hardwired control

Combinational/sequential logic directly generates control signals; it can be fast but harder to alter.

Microprogrammed control

A control store holds microinstructions that describe control steps; flexible sequencing may simplify complex instruction sets.

Micro-operations

A fetch sequence can transfer PC -> MAR, read memory, move data -> IR and increment PC.

Compare carefully

Speed and flexibility depend on the design; answer only using assumptions stated in the problem.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Do not infer clock speed solely from the control-unit category.

TRY FROM MEMORY

Cover this page. Explain Control unit: hardwired and microprogrammed using the picture, then give one example before looking again.

Memory hierarchy and performance

START WITH THE IDEA

Fast memory costs more per bit, so systems keep a small fast layer near the CPU and larger slower layers farther away. Locality makes this hierarchy effective.

A CONCRETE FIRST EXAMPLE

Hit time 1 ns, hit rate 0.9, miss penalty 50 ns beyond hit time: AMAT=1+0.1*50=6 ns.

PICTURE IT



IMAGINE IT THIS WAY

Keep the current tools on the desk (cache); keep bulk supplies in the storeroom (memory/storage).

THE IDEAS, ONE BY ONE

Hierarchy

Registers -> cache -> main memory -> storage typically grows in capacity and access time.

Locality

Temporal locality reuses recently accessed items; spatial locality accesses nearby addresses.

Performance

Hit rate is the fraction served at a level; miss rate=1-hit rate. Standard single-level AMAT=hit time+miss rate*miss penalty.

Memory interface

Address, data and control signals coordinate transfers between CPU, memory and devices.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Check whether the stated miss penalty already includes the initial hit time.

TRY FROM MEMORY

Cover this page. Explain Memory hierarchy and performance using the picture, then give one example before looking again.

Cache mapping

START WITH THE IDEA

A cache stores copies of memory blocks. Mapping determines where a block may live; the address is split into tag, index and block offset.

A CONCRETE FIRST EXAMPLE

32 KiB, 32-byte blocks gives 1024 lines. At 4-way associativity there are 256 sets; offset bits=5 and index bits=8.

PICTURE IT

1 Direct mapped → 2 Set associative → 3 Replacement and writes

IMAGINE IT THIS WAY

Direct mapped is one assigned locker; set associative is a small locker bank.

THE IDEAS, ONE BY ONE

Direct mapped

Each memory block has one possible cache line. Simple and fast, but conflicts can evict useful blocks.

Set associative

A block maps to one set and may occupy any of its ways. Fully associative is the special case of one set.

Address fields

Offset selects a byte within a block; index selects set; tag checks which memory block is present.

Replacement and writes

Associativity requires a replacement choice. Write-through updates lower memory immediately; write-back delays until eviction.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Compute lines before sets; do not confuse ways with number of sets.

TRY FROM MEMORY

Cover this page. Explain Cache mapping using the picture, then give one example before looking again.

Interrupts and DMA

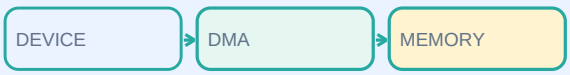
START WITH THE IDEA

I/O devices are slower and asynchronous. The CPU can poll, respond to interrupts, or configure DMA for block transfers.

A CONCRETE FIRST EXAMPLE

For a disk block, CPU programs DMA; controller copies bytes into memory; completion interrupt notifies the CPU.

PICTURE IT



IMAGINE IT THIS WAY

A bell asks the CPU for attention; DMA carries the whole tray while the CPU does other work.

THE IDEAS, ONE BY ONE

Polling

CPU repeatedly checks device status; simple, but wastes cycles while waiting.

Interrupt

Device signals the CPU when service or completion is needed; the CPU saves context and runs a handler.

DMA

A controller transfers a block between device and memory after CPU setup, reducing per-word CPU involvement.

Trade-off

Interrupts reduce waiting; DMA is useful for larger transfers but still requires setup and completion handling.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT DMA does not eliminate the CPU or all interrupts.

TRY FROM MEMORY

Cover this page. Explain Interrupts and DMA using the picture, then give one example before looking again.

Instruction pipelining and hazards

START WITH THE IDEA

Pipelining overlaps stages of several instructions to increase throughput. It does not remove the work required by one instruction.

A CONCRETE FIRST EXAMPLE

An ideal k-stage pipeline completes n instructions in n+k-1 cycles, assuming balanced stages, one instruction per cycle and no stalls.

PICTURE IT



IMAGINE IT THIS WAY

An assembly line overlaps jobs; one missing part can still stop a station.

THE IDEAS, ONE BY ONE

Stages

A textbook pipeline may use fetch, decode, execute, memory and writeback stages.

Structural hazard

Two stages need the same hardware resource at once.

Data hazard

An instruction needs a result not yet produced. Forwarding, stalls or scheduling may help.

Control hazard

Branches make the next PC uncertain; prediction or flushing handles wrong-path work.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT State assumptions before calculating speedup or cycle count.

TRY FROM MEMORY

Cover this page. Explain Instruction pipelining and hazards using the picture, then give one example before looking again.

Cache address fields and multi-level AMAT

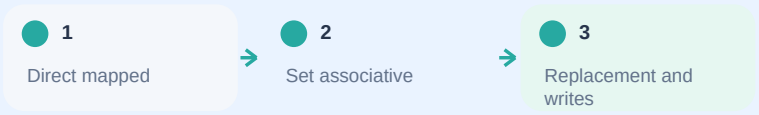
START WITH THE IDEA

A cache uses address bits to locate a set and block, then compares a tag. Average memory access time adds the cost of each miss path, weighted by its probability.

A CONCRETE FIRST EXAMPLE

With L1 hit time 1 ns, L1 miss rate 0.1, L2 hit time 5 ns, and L2 miss rate 0.2 with 50 ns memory: AMAT=1+0.1(5+0.2(50))=2.5 ns.

PICTURE IT



IMAGINE IT THIS WAY

A cache is a nearby shelf. AMAT counts the quick lookup plus the expected cost of going to farther shelves.

THE IDEAS, ONE BY ONE

Address split

Block offset bits = log2(bytes per block). Set index bits = log2(number of sets). Remaining address bits are tag.

Direct and set associative

A direct-mapped block has one possible set. A k-way cache has k candidate lines per set.

Two-level AMAT

AMAT = hit time L1 + miss rate L1 times (hit time L2 + miss rate L2 times main-memory penalty). Define rates conditionally.

Write policies

Write-through updates memory on each write; write-back marks a dirty line. Write-allocate fetches a missed block; no-write-allocate writes around the cache.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Use the L2 miss rate among L1 misses unless the question explicitly gives a global rate.

TRY FROM MEMORY

Cover this page. Explain Cache address fields and multi-level AMAT using the picture, then give one example before looking again.

IEEE 754 numbers, hazards and pipeline time

START WITH THE IDEA

Floating point encodes approximate real numbers; a pipeline overlaps instruction stages. Correct answers depend on bit fields, rounding and the exact hazard/forwarding assumptions.

A CONCRETE FIRST EXAMPLE

If stage delays are 180, 250, 150, 170 and 250 ns, with 10 ns latch overhead, the pipeline clock period is 260 ns.

PICTURE IT



IMAGINE IT THIS WAY

Think of the pipeline as an assembly line. A hazard is a worker reaching for a part that has not arrived.

THE IDEAS, ONE BY ONE

Special IEEE values

Exponent all zeros represents zero or subnormal values; exponent all ones represents infinity or NaN. Normal values use an implicit leading 1.

Precision and rounding

Finite significand bits mean many decimals are rounded. A tiny result may underflow; a large result may overflow to infinity.

Data hazards

RAW means a later instruction needs a result not yet written. Forwarding can bypass a value; load-use may still require a stall.

Pipeline timing

Clock period is set by the slowest stage plus latch overhead. Count fill/drain cycles, stalls and branch penalties as specified.

HOW TO USE IT ON A PROBLEM

Write the word size, representation and architecture assumptions. Trace data/control movement, compute address fields or cycles, and check units and overflow separately.

WATCH OUT Distinguish throughput after filling the pipeline from latency of one instruction.

TRY FROM MEMORY

Cover this page. Explain IEEE 754 numbers, hazards and pipeline time using the picture, then give one example before looking again.

Past-paper trail and coverage check

WHAT THE PAPERS MATCH

Verified anchors: 2018 Q23/Q34/Q50-Q51 (DRAM, cache mapping, pipeline and instruction encoding); 2022 Q17/Q33/Q54 (DMA and cache performance); 2023 Q32/Q41/Q63 (pipeline, memory interfacing, cache); 2024 CS1 Q15/Q30/Q53-Q54; 2025 CS2 Q28/Q32/Q39/Q49/Q55-Q56; 2026 CS1 Q14-Q16/Q36/Q38/Q60.

WHAT THIS EDITION ADDS

Added cache address derivation, nested AMAT, write policies, IEEE special values, RAW hazards, forwarding and pipeline clock-time calculation.

SYLLABUS BOUNDARY

Cache coherence and detailed multicore protocols are not named in the supplied 2027 COA syllabus.

HOW TO USE THIS CHECK

A question is a match when its tested skill is taught in the lesson pages, even if the story or numbers are new. Anchors are selected verified examples, not an exhaustive list; the separate Coverage and Gap Audit PDF identifies years whose scans could not be checked question by question.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

1. **Instructions and addressing modes**

   Without notes: define Instruction cycle and explain why this example works. Then recreate the steps on the lesson page.

2. **ALU and arithmetic**

   Without notes: define Shifts and explain why this example works. Then recreate the steps on the lesson page.

3. **Control unit: hardwired and microprogrammed**

   Without notes: define Compare carefully and explain why this example works. Then recreate the steps on the lesson page.

4. **Memory hierarchy and performance**

   Without notes: define Memory interface and explain why this example works. Then recreate the steps on the lesson page.

5. **Cache mapping**

   Without notes: define Replacement and writes and explain why this example works. Then recreate the steps on the lesson page.

6. **Interrupts and DMA**

   Without notes: define Trade-off and explain why this example works. Then recreate the steps on the lesson page.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

7 **Instruction pipelining and hazards**

Without notes: define Control hazard and explain why this example works. Then recreate the steps on the lesson page.

8 **Cache address fields and multi-level AMAT**

Without notes: define Write policies and explain why this example works. Then recreate the steps on the lesson page.

9 **IEEE 754 numbers, hazards and pipeline time**

Without notes: define Pipeline timing and explain why this example works. Then recreate the steps on the lesson page.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

1. **LESSON 1: Instructions and addressing modes**

   For LOAD R1, 8(R2), effective address=contents(R2)+8; memory at that address is loaded into R1.

2. **LESSON 2: ALU and arithmetic**

   Four-bit signed 7+3: 0111+0011=1010. Positive + positive produced a negative-looking result, so signed overflow occurred.

3. **LESSON 3: Control unit: hardwired and microprogrammed**

   During fetch, PC identifies the next instruction address; after memory returns the instruction, it is stored in IR for decode.

4. **LESSON 4: Memory hierarchy and performance**

   Hit time 1 ns, hit rate 0.9, miss penalty 50 ns beyond hit time: AMAT=1+0.1*50=6 ns.

5. **LESSON 5: Cache mapping**

   32 KiB, 32-byte blocks gives 1024 lines. At 4-way associativity there are 256 sets; offset bits=5 and index bits=8.

6. **LESSON 6: Interrupts and DMA**

   For a disk block, CPU programs DMA; controller copies bytes into memory; completion interrupt notifies the CPU.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

7. LESSON 7: Instruction pipelining and hazards

An ideal k-stage pipeline completes n instructions in n+k-1 cycles, assuming balanced stages, one instruction per cycle and no stalls.

8. LESSON 8: Cache address fields and multi-level AMAT

With L1 hit time 1 ns, L1 miss rate 0.1, L2 hit time 5 ns, and L2 miss rate 0.2 with 50 ns memory: AMAT=1+0.1(5+0.2(50))=2.5 ns.

9. LESSON 9: IEEE 754 numbers, hazards and pipeline time

If stage delays are 180, 250, 150, 170 and 250 ns, with 10 ns latch overhead, the pipeline clock period is 260 ns.

Your practice and reference library

01 **Beginner lessons**

Plain-language explanation, visual sequence, example and watch-outs for every listed 2027 topic cluster.

02 **Recall and GATE-style practice**

The illustrated recap section includes questions and a separate answer explanation.

03 **Detailed reference**

Your supplied subject handbook and chapter PDFs follow for extra depth, reading and worked material.

Included source documents

Illustrated recall and GATE-style practice	COA - Computer Organization and Architecture - GATE 2027 Illustrated Notes.pdf	12 pages
Detailed supplied reference	COA.pdf	59 pages
Detailed supplied reference	Computer-Organization-and-Architecture-GATE-2027-Master-Sheet.pdf	10 pages

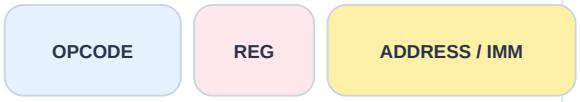
The official 2027 syllabus map precedes the lessons. Original reference pages are retained so you can use them for slower reading and extra practice.

Computer Organization and Architecture

GATE 2027 | VISUAL LEARNING NOTES | STARTING FROM THE BASICS

Follow one instruction from bits to results, then to memory.

PICTURE IT



bits tell the CPU what to do and where data lives

HOW TO USE THIS NOTEBOOK

1. **Learn the idea**
   Read the plain-language explanation before memorizing a formula.
2. **See it**
   Study the labeled picture and redraw it from memory.
3. **Follow a solve**
   Work through the examples step by step.
4. **Test yourself**
   Attempt the GATE-style questions before opening the key.

THE PATH YOU WILL FOLLOW

- Instruction execution
- Memory and I/O
- Pipelining

IF YOU FEEL COMPLETELY NEW

That is okay. Start with the first topic page, keep a pencil beside you, and copy each diagram once. Do not jump straight to the answer key. The questions are meant to come after the concepts.

Your route through this subject

Follow the topics in order. Each page teaches one idea, shows a picture, gives a recall cue and points out a common trap.

INSTRUCTION EXECUTION

- [ ] Instruction set and addressing modes.
- [ ] Design of arithmetic and logic unit (ALU).
- [ ] Design of control unit: hardwired and microprogrammed.

MEMORY AND I/O

- [ ] Memory interfacing and hierarchy: performance and cache memory mapping.
- [ ] I/O interface: interrupts and DMA.

PIPELINING

- [ ] Instruction pipelining.
- [ ] Pipeline hazards.

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

Tick a topic only after you can explain its picture without looking. A tick means "I can teach it," not "I have seen it."

BEFORE EACH STUDY SESSION

Pick one topic. Read its idea, redraw its diagram, follow the worked examples, then answer questions with the notes closed. Circle only the step where you got stuck - that is your next revision target.

CONCEPT 01 / COA

Instruction set and addressing modes

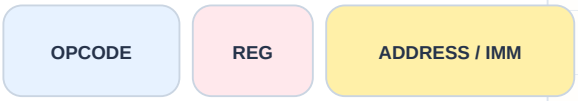
START WITH THE IDEA

An instruction has operation and operand fields. Addressing mode says where an operand is found; effective address may be literal, register-based, direct, indexed or PC-relative. Read the exact convention given.

A CONCRETE FIRST EXAMPLE

An instruction is a compact command: the opcode selects the operation, while register or address fields locate operands. Addressing modes describe how an operand is found, such as immediate, direct, indirect or indexed. For LOAD R1, 8(R2), the effective address is contents(R2)+8; the value at that address is copied to R1.

PICTURE IT



bits tell the CPU what to do and where data lives

IMAGINE IT THIS WAY

The opcode is a food order; addressing mode is how the cook finds the ingredient.

HOW TO USE IT ON A PROBLEM

For every operand, label value vs address vs address-of-address.

WATCH OUT

Immediate mode uses the value in the instruction; it does not fetch that operand from memory.

TRY FROM MEMORY

Cover the page and explain Instruction set and addressing modes using the diagram. Give one example before looking again.

CONCEPT 02 / COA

ALU and arithmetic

START WITH THE IDEA

The ALU performs arithmetic/logic operations. Adders use propagate/generate ideas; signed overflow depends on operand/result signs. Shifts and comparison operations often reuse arithmetic hardware.

A CONCRETE FIRST EXAMPLE

The ALU performs integer arithmetic and logic. Two's-complement addition uses ordinary binary addition; subtraction A-B is A+NOT(B)+1. Signed overflow is different from carry-out. 4-bit signed 7+3 is 0111+0011=1010. The result looks negative, so positive-plus-positive caused signed overflow.

PICTURE IT



control signals move data between registers and ALU

IMAGINE IT THIS WAY

The ALU is a multi-tool; control bits choose which blade is active.

HOW TO USE IT ON A PROBLEM

Track width, signedness and carry/overflow separately.

WATCH OUT

Carry-out and signed overflow are different flags.

TRY FROM MEMORY

Cover the page and explain ALU and arithmetic using the diagram. Give one example before looking again.

CONCEPT 03 / COA

Control unit design

START WITH THE IDEA

Hardwired control uses logic circuits and is typically fast but harder to modify. Microprogrammed control sequences microinstructions from a control store, making complex control easier to change.

A CONCRETE FIRST EXAMPLE

The control unit turns an instruction into timed control signals. Hardwired control uses logic circuits; microprogrammed control sequences stored microinstructions. Both coordinate register transfers, ALU work and memory access. A fetch cycle can be read as: PC -> MAR, memory read, memory data -> IR, then PC increments.

PICTURE IT



control signals move data between registers and ALU

IMAGINE IT THIS WAY

Hardwired control is a fixed circuit; microcode is a recipe book read one step at a time.

HOW TO USE IT ON A PROBLEM

Compare implementation, speed and flexibility under the question's assumptions.

WATCH OUT

Do not infer performance without an architecture or timing model.

TRY FROM MEMORY

Cover the page and explain Control unit design using the diagram. Give one example before looking again.

CONCEPT 04 / COA

Memory hierarchy and performance

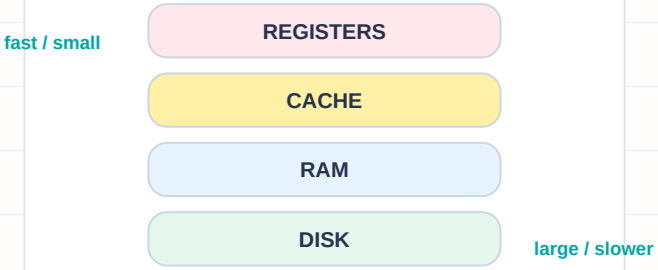
START WITH THE IDEA

Registers/cache are small and fast; main memory is larger and slower; storage is much larger and slower. Locality makes a cache useful. AMAT = hit time + miss rate x miss penalty (under the standard single-level model).

A CONCRETE FIRST EXAMPLE

A memory hierarchy keeps frequently used data close to the CPU. Registers are smallest and fastest; cache and RAM trade speed for capacity; storage is larger but slower. Locality makes this useful. If a loop reads arr[0], arr[1], arr[2] in sequence, spatial locality makes nearby cache lines useful.

PICTURE IT



IMAGINE IT THIS WAY

Keep today's tools on the desk (cache); archive the rest in a storeroom (memory/storage).

HOW TO USE IT ON A PROBLEM

Convert hit rate to miss rate: miss rate=1-hit rate.

WATCH OUT

Check whether the stated miss penalty includes or excludes hit time.

TRY FROM MEMORY

Cover the page and explain Memory hierarchy and performance using the diagram. Give one example before looking again.

CONCEPT 05 / COA

Cache mapping

START WITH THE IDEA

Direct mapped: one possible line per block. Fully associative: any line. Set associative: one of W ways in one indexed set. Address splits into tag, index and block offset.

A CONCRETE FIRST EXAMPLE

A cache maps each memory block to a cache location. Direct mapping has one possible line; set associative mapping offers several ways; fully associative mapping allows any line. Hit time and miss penalty both matter. A 32 KiB cache with 32-byte lines has 1024 lines. At 4-way associativity, it has 1024/4=256 sets.

PICTURE IT



address bits select a place, then a byte

IMAGINE IT THIS WAY

A direct-mapped cache has one locker per key; set-associative has a small locker bank; fully associative searches every locker.

HOW TO USE IT ON A PROBLEM

Lines = capacity/block size; sets = lines/ways; offset bits=log2(block size).

WATCH OUT

Use bytes and powers of two consistently; distinguish line count from set count.

TRY FROM MEMORY

Cover the page and explain Cache mapping using the diagram. Give one example before looking again.

CONCEPT 06 / COA

I/O, interrupts and DMA

START WITH THE IDEA

Polling repeatedly checks device status. Interrupts let a device signal the CPU. DMA transfers a block between device and memory with reduced CPU involvement.

A CONCRETE FIRST EXAMPLE

An interrupt asks the CPU to respond to an event; DMA lets a controller transfer a block between device and memory with fewer CPU data moves. Interrupts usually signal completion or need for service. For a disk block, the CPU starts DMA, the controller copies the block, then raises an interrupt when it finishes.

PICTURE IT



control signals move data between registers and ALU

IMAGINE IT THIS WAY

A waiter can be polled every minute, ring a bell, or carry a tray directly to the table (DMA).

HOW TO USE IT ON A PROBLEM

Choose based on transfer size, CPU work and latency constraints.

WATCH OUT

Interrupt-driven I/O is not the same as DMA; DMA still needs control setup.

TRY FROM MEMORY

Cover the page and explain I/O, interrupts and DMA using the diagram. Give one example before looking again.

CONCEPT 07 / COA

Pipelining and hazards

START WITH THE IDEA

Pipelines overlap instruction stages. Structural hazards compete for hardware; data hazards depend on values; control hazards follow branches. Forwarding, stalls and prediction can mitigate them. Ideal k-stage time for n instructions is about (k+n-1) cycles.

A CONCRETE FIRST EXAMPLE

Pipelining overlaps instruction stages to improve throughput. It does not make one instruction's work disappear. Structural, data and control hazards may stall or redirect the pipeline. With 5 stages, an ideal long pipeline approaches one completed instruction per cycle after filling; a branch stall breaks that ideal.

PICTURE IT



Different instructions overlap in time

IMAGINE IT THIS WAY

An assembly line overlaps jobs, but one missing part can stall the next station.

HOW TO USE IT ON A PROBLEM

Draw stage-by-cycle tables for short sequences.

WATCH OUT

Ideal speedup assumes balanced stages and no hazards.

TRY FROM MEMORY

Cover the page and explain Pipelining and hazards using the diagram. Give one example before looking again.

Let's solve it together

Cover the answer with your hand. Try each step first, then compare your reasoning.

EXAMPLE 1: Cache geometry

QUESTION

A 2 KiB cache is 2-way set associative with 16-byte blocks. Find lines and sets.

WORK THROUGH THE STEPS

Capacity = 2048 bytes. Lines = 2048/16 = 128. Sets = 128/2 = 64. Offset bits = log2(16)=4; index

ANSWER 128 lines, 64 sets, 4 offset bits and 6 index bits.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

EXAMPLE 2: Average memory access time

QUESTION

Hit time is 1 ns, hit rate is 0.9, and miss penalty is 50 ns beyond hit time. Find AMAT.

WORK THROUGH THE STEPS

Miss rate = 1-0.9=0.1. AMAT = 1 + 0.1 x 50 = 6

ANSWER 6 ns.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

Now you try

Attempt these without the notes. The question types are marked MCQ, MSQ or NAT. Answers and explanations are on the next page.

MCQ In immediate addressing, the operand is:

(A) At an address in memory (B) In the instruction (C) In the next cache line (D) Always in a register

NAT A 32 KiB cache has 32-byte blocks and is 4-way set associative. How many sets?

Enter an integer.

MCQ Which condition signals signed addition overflow?

(A) Carry-out is 1 (B) Same-sign operands produce opposite-sign result (C) Result is zero (D) Low bit is 1

NAT An ideal 5-stage pipeline executes 20 instructions with no stalls. How many cycles?

Assume one instruction enters each cycle.

MSQ Which statements are true of DMA?

(A) CPU initializes the transfer (B) Device data can move to memory without CPU copying every word (C) DMA eliminates all interrupts (D) Completion may be signaled by an interrupt

Exam habit: decide what is being asked before doing arithmetic. Show units, assumptions and one line of reasoning.

Answers with the why

Read the explanation even when your answer is correct. The goal is to learn the reasoning pattern.

1. B

WHY

The instruction contains the literal operand value.

2. 256

WHY

Lines=32768/32=1024; sets=1024/4=256.

3. B

WHY

For two's-complement addition, overflow occurs when same-sign operands yield opposite-sign result.

4. 24

WHY

n+k-1=20+5-1=24.

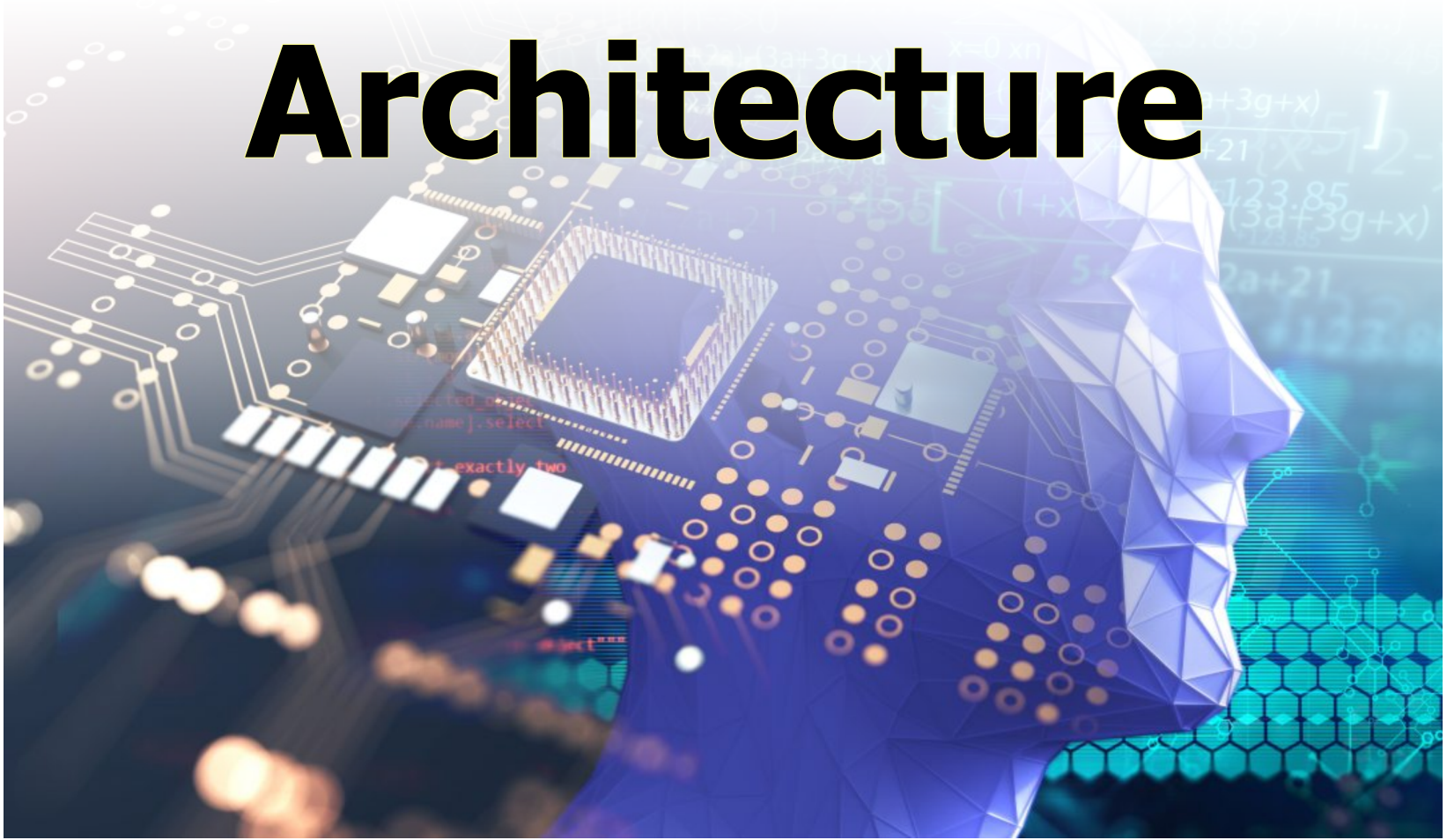
5. A, B, D

WHY

DMA offloads the bulk transfer but setup and completion handling still involve the CPU.

Review loop: mark each missed concept page, redraw its picture, and retry a similar problem tomorrow.

Computer Organization & Architecture



Published By:

Physics Wallah

ISBN: 978-93-94342-39-2

Mobile App: Physics Wallah (Available on Play Store)

Website: www.pw.live

Email: support@pw.live

Rights

All rights will be reserved by Publisher. No part of this book may be used or reproduced in any manner whatsoever without the written permission from author or publisher.

In the interest of student's community:

Circulation of soft copy of Book(s) in PDF or other equivalent format(s) through any social media channels, emails, etc. or any other channels through mobiles, laptops or desktop is a criminal offence. Anybody circulating, downloading, storing, soft copy of the book on his device(s) is in breach of Copyright Act. Further Photocopying of this book or any of its material is also illegal. Do not download or forward in case you come across any such soft copy material.

Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

This book and the individual contribution contained in it are protected under copyright by the publisher.

(This Module shall only be Used for Educational Purpose.)

INDEX

1. Machine Instructions ..... 4.1 – 4.5
2. Addressing Modes ..... 4.6 – 4.8
3. ALU Data Path ..... 4.9 – 4.13
4. CPU Control Unit Design ..... 4.14 – 4.19
5. Memory Interfacing ..... 4.20 – 4.22
6. Input Output Interfacing ..... 4.23 – 4.28
7. Pipelining ..... 4.29 – 4.43
8. Cache Memory ..... 4.44 – 4.48
9. Main Memory ..... 4.49 – 4.51
10. Secondary Storage ..... 4.52 – 4.56

1 MACHINE INSTRUCTIONS

1.1 Introduction

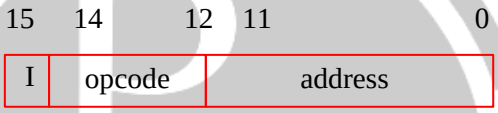
Opcode – Specifies the operation code. The number of bits in opcode depends on total operations.
Address – Specifies the operand address.

- The basic computer has three instruction code formats.

(i) **Memory – Reference instruction**

I = O ⇒ Direct addressing

I = 1 ⇒ Indirect addressing



Instruction format

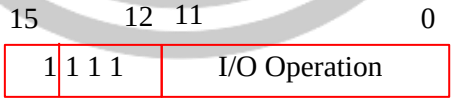
(opcode = 000 to 110)

(ii) **Register – Reference instruction**



(opcode = 111, I = 1)

(iii) **Input – output Instruction**



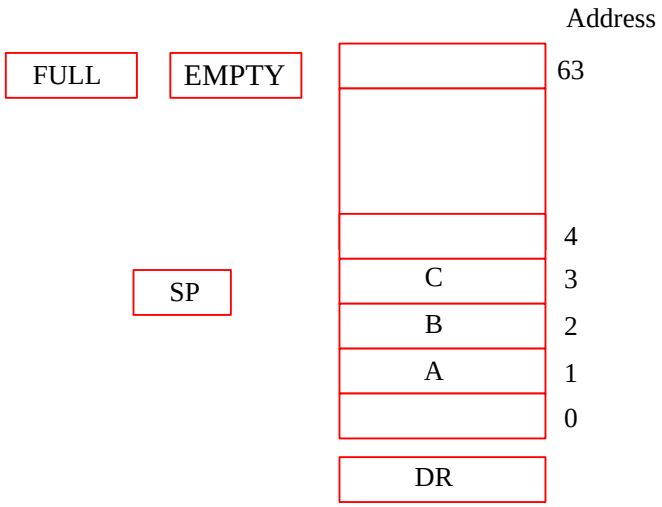
(op code = 111, I = 1)

A stack can be placed in a portion of a large memory or it can be organized as a collection of a memory words or registers. i.e., it may be

(i) Register stack
(ii) Memory stack

1.1.1. Register Stack

- The following figure shows the organization of a 64-word register stack. The stack pointer register SP contains a binary number whose value is equal to the address of the word that is currently on top of stack.
- Initially SP is cleared, EMPTY is set to 1, FULL is cleared to 0. If the stack is not full (if FULL = 0), a new item is inserted. With a push operation.



PUSH:

SP ← SP + 1 [increment stack pointer]
M{SP} ← DR [write item on top of stack]
If (SP = 0) then FULL ←1 [check if stack is full]
EMTY ← 0 [mark the stack not empty]

- A item is deleted from the stack if the stack is not empty (if EMTY = 0), called POP operation

DR ← M{SP} [Read item from top of stack]
SP ← Sp – 1 [Decrement stack pointer]
If (SP = 0) then EMTY – 1

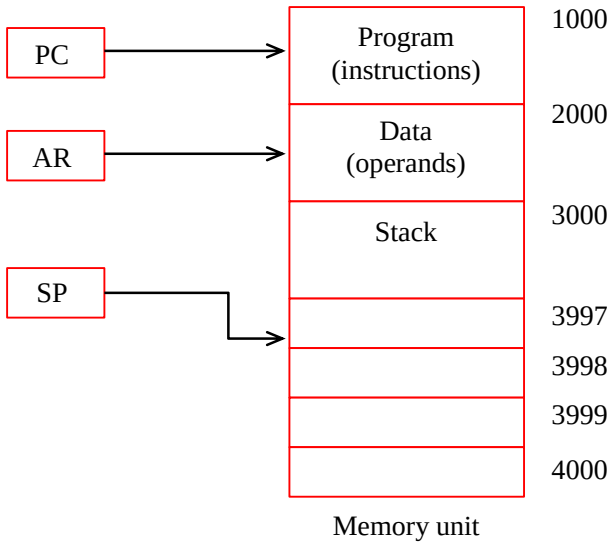
1.1.2. Memory Stack

- A stack can be implemented in a random-access memory. The stack can be implemented by assigning a portion of memory to a stack operation and using a processor register as a stack pointer. i.e.,
- A new item is inserted with the push operation as follows:

PUSH: SP ← SP – 1
M[SP] ← DR

- A new item is deleted with a POP operation as follows

DR ← M[SP]
SP ← SP + 1



- A stack pointer is loaded with an initial value. This initial value must be the bottom address of an assigned stack in memory. SP is automatically incremented or decremented with every **PUSH** or POP operation.

1.1.3. Reverse Polish Notation

- An expression in post fix form is often called reverse polish notation.

The infix expression (A + B) * [C*(D + E) + F] in reverse polish notation as AB + DE + C * F + *

- A reverse polish expression can be implemented or evaluated using stack for the expression

  X ← (A + B) * (C + D)
  With values A = 2, B = 3, C = 4, d = 2 is
  X ← (2 + 3) * (4 + 2) ← Infix expression
  x ← 23 + 42 + * ← Reverse polish expression

1.2 Instruction Types

- The basic unit of program is the instruction.

  Instructions are classified based on

  (1) Opcode – called functional classification

  (2) Based on number of references.

1.3 Functional Classification

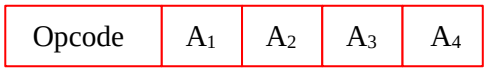
- Data transfer instructions – (LOAD, STOR, MOV, EXCT, IN, OUT, PUSH, POP)
- Arithmetic instructions – (INC, DEC, ADD, SUB, MUL, DIV)
- Logical Instructions & Shift instructions. (CLR, COM, AND, XOR, OR, SHR, SHL, SHRA, SHLA, RO, ROL….)
- Branching instructions. (ABR, JMP, SKP, CALL, RETURN)

1.4 Based on Number of References

Based on number of references, the instructions can be classified as

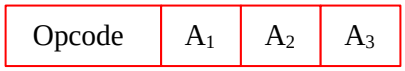
- 4 – address instructions
- 3- address instructions
- 2 – address instructions
- 1 – address instructions
- 0 – address instructions
- RISC Instructions

(i) 4 – Address Instructions:



- A_1, A_2 refers operands
- A_3 refers destination
- A_4 next instruction address.
- Since A_4 is like program counter, the processor which supports 4-address instructions need not use PC.
- The length of instruction is more, hence requires more than one reference.

(ii) 3 – address Instructions



- Each address field specify a register or memory operand.
- It results in short program when evaluating arithmetic expressions.
- Requires too many bits to specify three addresses

Example: To evaluate X = (A + B) * (C + D)

Add R_1, A, B $R_1 \leftarrow M[A] + M[B]$

Add R_2, C, D $R_2 \leftarrow M[C] + M[D]$

MUL X, R_1, R_2 $M[X] \leftarrow R_1 * R_2$

For these instructions program counter must be required.

(iii) 2 – address Instructions:



- A_1 – first operand and destination
- A_2 – second operand
- Uses MOV instruction for data transfer

Example:

$X \leftarrow (A + B) * (C + D)$

MOV R_1, A

ADD R_1, B

MOV R_2, C

ADD R_2, D

MUL R_1, R_2

MOV X R_1

One of the operand permanently lost.

(iv) 1 – address Instructions



- Uses an implied accumulator (AC) register for all data manipulation.
- Easily decoded and processed instructions.

Example: X ← (A + B) * (C + D)

LOAD	A	AC ← M[A]
ADD	B	AC ← AC + M[B]
STOR	T	M[T] ← AC
LOAD	C	AC ← M[C]
ADD	D	AC ← AC + M[D]
MUL	T	AC ← AC * M[T]
STOR	X	M[X] ← AC

(v) Zero – address Instructions:



- The operands are referenced complexity from stack.
- More complex approach than others.
- Any changes in order of operands effect the result.

Example: X = (A + B) * (C + D)

```
PUSH    A
PUSH    B
ADD
PUSH    C
PUSH    D
ADD
MUL
POP     X
```

(vi) RISC Instructions:

- All instructions are executed with in the registers of CPU, without referring to memory (Except LOAD, STOR)
- Uses in RISC processor
- LOAD & STOR are used between data transfer.

Example: X = (A + B) * (C + D)

```
LOAD    R1, A
LOAD    R2, B
LOAD    R3, C
LOAD    R4, D
ADD     R1, R1, R2
ADD     R3, R3, R4
MUL     R1, R1, R3
STOR    X, R1
```

❐❐❐

2 ADDRESSING MODES

2.1 Introduction

The various addressing modes commonly used are:

Implied Mode:

In this mode the operands are specified implicitly in the definition of the instruction.

Example:

- Complement Accumulator (CMA) [All register reference instructions that use an accumulator are implied - mode instructions]
- Zero-address instructions in a stack organized computer. [here the operands are implied to be on top of the stack]

Immediate Mode:

In this mode the operand is specified in the instruction itself.

- It is very faster
- Useful for initializing register to a constant value.

Example:

ADD 10 i.e., AC ← AC + 10

- The range of value limited by the address field

Register Mode:

In this mode the operands are in registers that reside within the CPU.

- A K-bit field can specify any one of 2^k registers.
- Reduces the length of the address field.

Example:

ADD R1

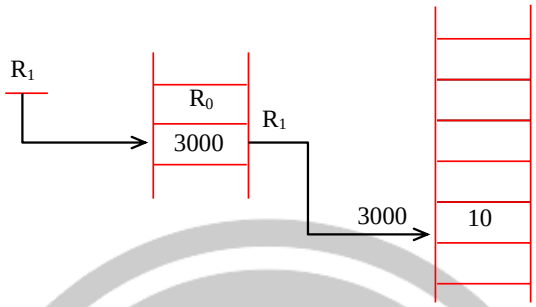
Register Indirect Mode:

In this mode the instruction specifies a register in the CPU whose contents give the address of an operand in memory.

- Before using this mode, the programmer must ensure that the memory address of the operand is placed in the processor register with a previous instruction.
- The advantage is the address field uses fewer bits to select or register.

Example:

ADD (R1)

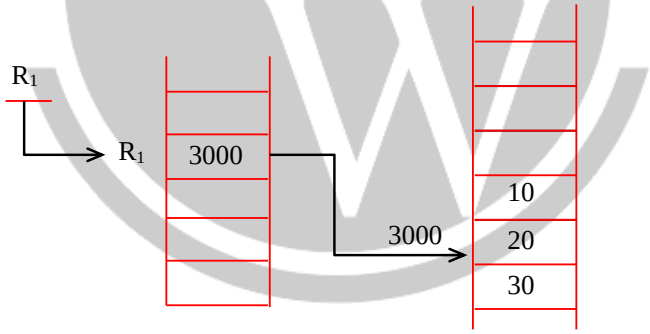


Autoincrement or Autodecrement Mode:

It is similar to register indirect mode except that the register is incremented or decremented after its value is used to access memory.

Example:

ADD (R_1)

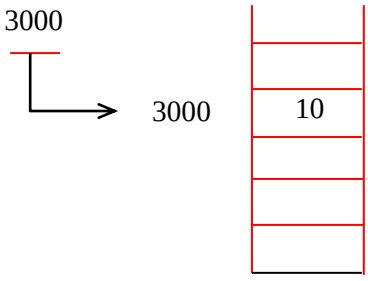


Direct Addressing Mode:

In this mode the effective address is equal to the address part of the instructions.

- The operand resides in memory and its address is given directly by the address field of the instruction.
- Used to represent global variables in a program.
- In a branch type instruction, the address field specifies the actual branch address space.
- Allows to access limited address space.
- The effective address is defined to be the memory address obtained from the computation dictated by the given addressing mode.
- The direct addressing mode is also called "Absolute mode".

Example. ADD 3000



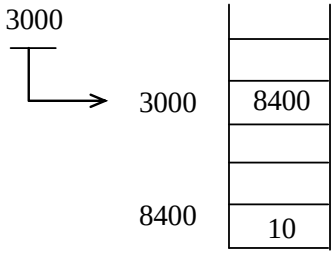
i.e. AC ← AC + M [3000]

Indirect Addressing mode:

In this mode, the address field of the instruction gives the address where the effective address is stored in memory.

- Allows to access larger address space
- Requires 2 memory cycles to read an operand

Example ADD 3000



2.1.1 Displacement addressing modes

The address field of the instruction added to the content of a specific register in the CPU. i.e,
Effective Address = Address part of instruction + content of Register.

2.1.2 The Various displacement addressing modes are

Relative addressing mode:

In this mode the content of program counter (PC) is added to the address part of the instruction.

- The address part is a signed number (2's complement) that can be either positive or negative.
- The result produces effective address whose position in memory is relative to the address of the next instruction.

EA = Address Part (off set) + PC value

- Used with branch-type instructions when the branch address is in the area surrounding the instruction word itself.
- The advantage is, address field can be specified with a small number of bits compared to direct address.

Indexed Addressing mode:

In this mode the content of an index register is added to the address part of the instruction.

- Index register contains index value.
- Address part specifies the beginning address of a data array in memory.

EA = Address Part (base address of data array) + Index register value (index value)

- Used for accessing array of data.
- The index register can be special CPU register or any general purpose register.

Base register Addressing mode:

In this mode the content of a base register is added to the address part of the instruction.

- The base register is assumed to hold base address.
- The address field gives the displacement relative to this base address.

EA = Address Part (displacement/offset) + Base register value (Base address)

❐❐❐

3 ALU DATA PATH

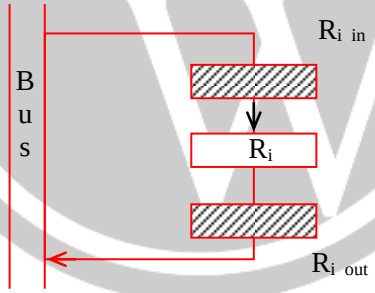
3.1 Introduction

- The sequence of operations involved in processing an instruction constitutes an instruction cycle.
  The instruction cycle consists of phases like.
  (1) Fetch cycle (2) Decode cycle
  (3) Operand fetch cycle (4) Execute cycle
- To perform these, the processor unit has to perform set of operations called “Micro-Operations”.

3.2 The Basic Operations Performed are

3.2.1 Register transfers:

- In general, the input & output of register R_i are connected to the bus via switched controlled by the signals R_i in and $R_i$out



- When Ri in is set to 1, the data on the bus are loaded into Ri.
- When Ri out is set to 1, the contents of register Ri are placed on the bus.

- To transfer the contents of R_1 to register R_4 : $R_4 \leftarrow R_1$.
- Enable the output of register R1 by setting R1 out to 1. This places the contents of R1 on the processor bus.
- Enable the input to register R_4 by setting $R_{4\,in}$ to 1. This loads data from the processor bus into register R_4.

3.2.2 Performing ALU operations

The ALU has two inputs A and B and one output

- A gets operand from output of MUX
- B gets operand from bus
- The result is gets stored in Z-register.

The sequence of steps for ALU operation $R_3 \leftarrow R_1$

(1) The contents of R_1 are loaded in Y.

(2) The contents of Y_1R_2 are applied to A & B inputs of ALU, performs ALU operation & stores the result in Z-register.

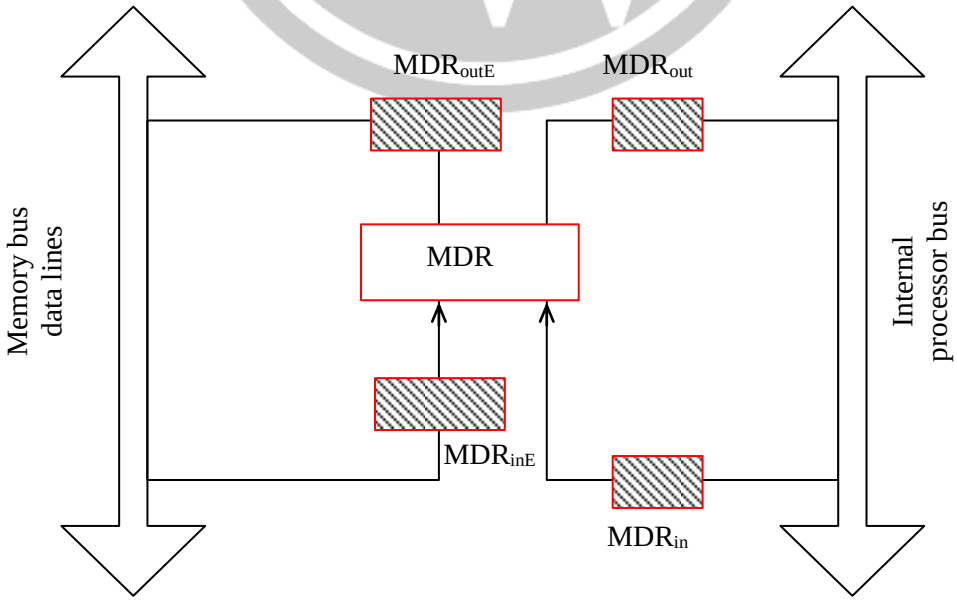
(3) The contents of Z are stored in R_3.

- The sequence of operations is
  - (1) $R_{1\ out}$, Y_{in}
  - (2) $R_{2\ out}$, select Y, Add, Z_{in}
  - (3) Z_{out}, $R_{3\ in}$
  - The functions performed by ALU depends on the signals applied to its control lines. (Here Add line is set to 1).
  - Only one register output can be connected to the bus during any clock cycle.
  - The no of steps indicates no of clocks.

3.2.3 Fetching a word from memory (read operation)

- To read a memory word, consider MOV (R_1), R_2. The action needed to execute this instruction are
  - (1) MAR $\leftarrow$ [R_1]
  - (2) Start a read operation on the memory bus
  - (3) Wait for the MFC (Memory function to complete) response from memory.
  - (4) Load MDR from the memory bus
  - (5) $R_2 \leftarrow$ [MDR]

- Hence the memory read operation sequence of operations is
  - (1) $R_{1\ out}$, MAR_{in}, Read
  - (2) WMFC (wait for memory operation to complete), $MDR_{\ inE}$
  - (3) MDR_{out}, $R_{2\ in}$



3.2.4 Storing a word in Memory (write operation)

- The sequence of steps for write operation MOV R_2,
  (1) The desired address is loaded into MAR
  (2) The data to be written is loaded into MDR
  (3) Write command is issued.
  (4) Wait for memory operation to complete.
- The sequence of operations is
  (1) $R_{1\ out}$, MAR $_{in}$
  (2) R_{2out}, MDR_{in}, write
  (3) MDR_{outE}, WMFC

3.2.5 Branch Instructions

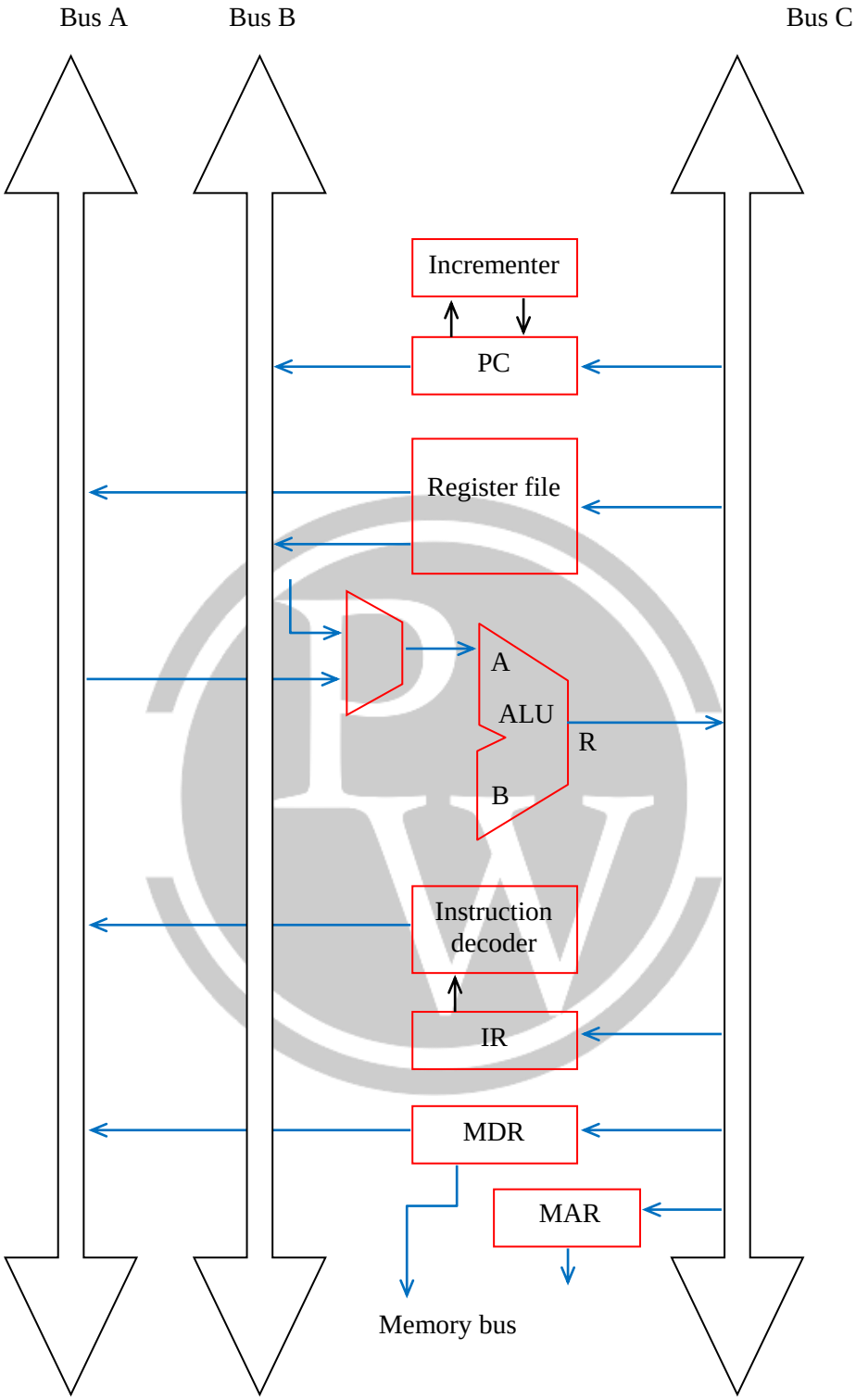
- A branch instruction replaces the content of PC with the branch target address. The address is obtained by adding an offset X, which is given in the branch instruction, to the updated value of PC.
- The control sequence for branching (unconditional is
  (1) PC_{out}, PC_{in}, Y_{in}, WMFC
  (2) Z_{out}, PC_{in}, Y_{in}, WMFC
  (3) MDR_{out}, IR_{in}
  (4) Offset – of – IR_{out}, Add, Z_{in}

3.2.6 Execution of complete Instruction

- Executing an instruction requires (Add (R_3), R_1)
  (1) Fetch the instruction
  (2) Fetch the first operand (memory location specified by R_3)
  (3) Perform the addition
  (4) Load the result into R_1
- The control sequence for the execution of ADD (R_3), R_1 in a single-bus organization is
  (1) PC_{out}, MAR_{in}, Read, Select, Add, Z_{in}
  (2) Z_{out}, PC_{in}, Y_{in}, WMFC
  (3) MDR_{out}, IR_{in}
  4) R_{3out}, MAR_{in}, Read
  (5) $R_{1\ out}$, Yin, WMFC
  (6) MDR_{out}, select Y, Add, Z_{in}
  (7) Z_{out}, $R_{1\ in}$, End

3.3 Multiple – Bus Organization

- With simple single bus structure, the resulting control sequence is quite long because only one data item can be transferred over the bus in a clock cycle. To reduce the number of steps needed, most commercial processors provide multiple internal paths that enable several transfers to take place in parallel.
- The three bus organization of data path is

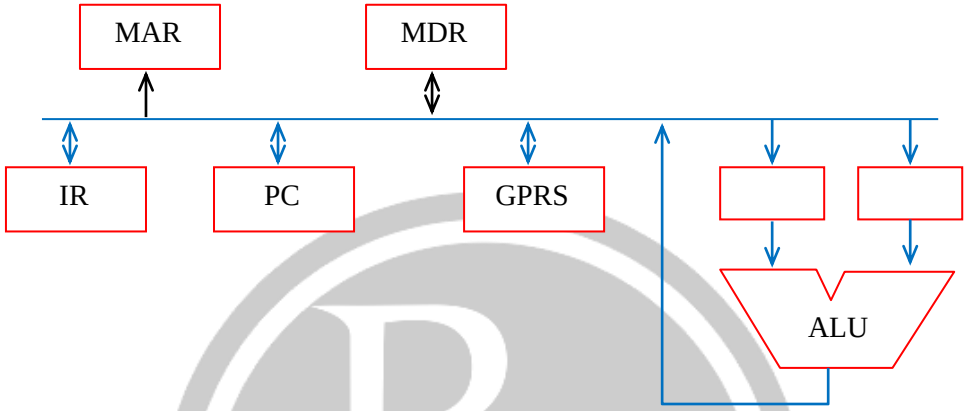


- All general – purpose registers are combined into a single block called register file. It has two – outputs, allowing the contents of two different registers to be accessed simultaneously and have their contents placed on buses A and B. The data on bus 'C' to be loaded into third register during same clock.

- Buses A and B are used to transfer the source operands to the A and B inputs of ALU, the result is transferred to the destination over bus C.

- The control sequence for instruction ADD R_4, R_5, R_6
  (1) P_{Cout}, R = B, MAR_{in}, Read, Inc PC
  (2) WMFC
  (3) MDT_{out} B, R = B, IRin
  (4) $R_{4\ outA}$, $R_{5\ outB}$, select A, Add, $R_{6\ in}$, end.

Example (1)



The ALU, bus & registers in data path are of identical size. All operations including incrementing of PC and GPRSs are to be carried out in the ALU. 2-clock cycles are needed for memory read operation. (one for loading address in MAR and loading data from memory into the MDR).

❑❑❑

4 CPU CONTROL UNIT DESIGN

4.1 Introduction

To execute instructions, the processor must have some means of generating the control signals needed in the proper sequence. The two approaches used for this purpose are

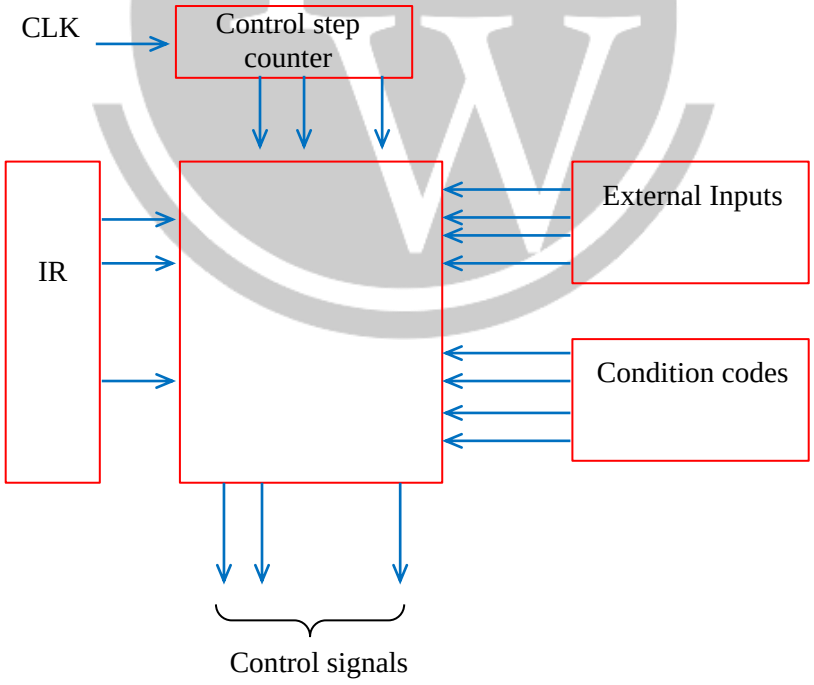
(1) Hardwired control

(2) Micro programmed control

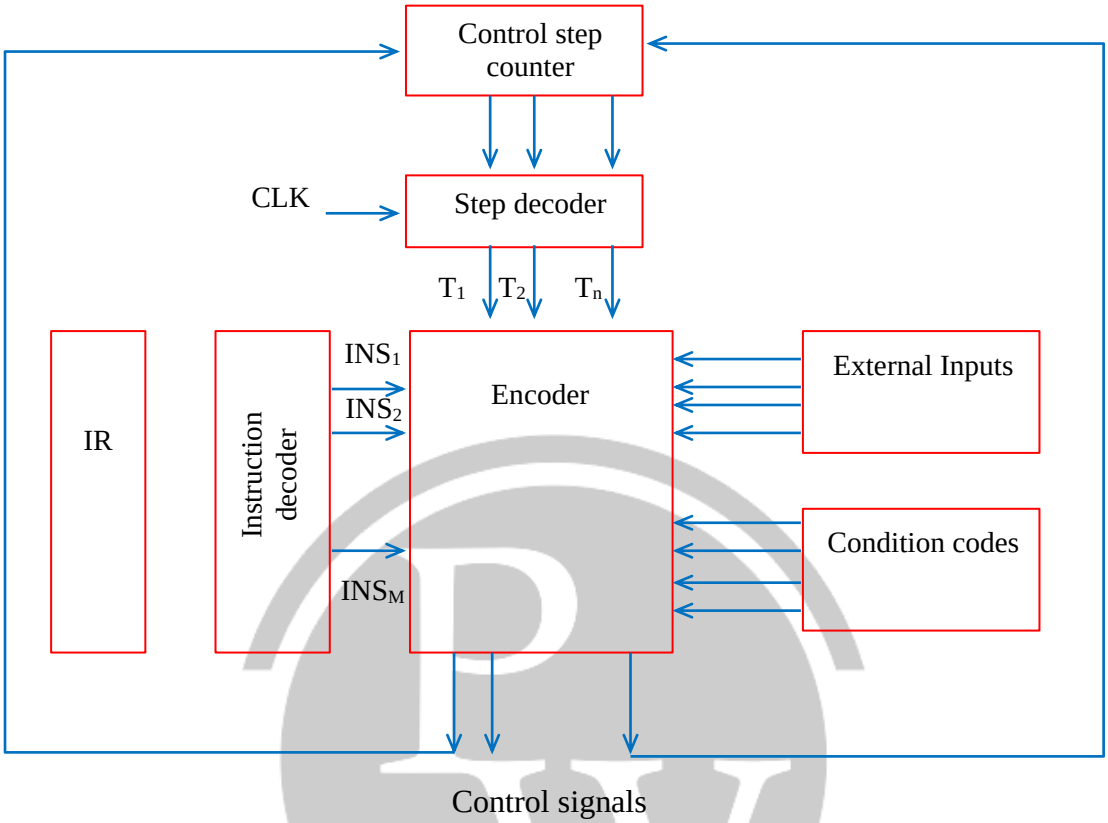
The purpose of control unit is to generate accurate timing & control signals.

4.1 Hardwired Control

- The control unit uses fixed logic circuits to interpret instructions and generate control signals from them. Every control signal is expressed as SOP (sum of products) expression and realized using digital hardware.



- The below figure shows the detailed hardwired control unit design.



- For executing an instruction completely each step is completed in one clock period. A counter is used to keep track of the control steps.
- The required control signals are determined with the following information.
  - (1) Contents of control step counter.
  - (2) Contents of instruction register
  - (3) Contents of condition code flags
  - (4) External input signals, such as MFC and interrupt requests.
  - The instruction decoder decodes the instruction loaded in IR
  - The step decoder provides a separate signal line for each step or time slot in a control sequence.
  - The encoder gets input from instruction decoder step decoder, external inputs and condition codes, thus uses to generate the individual control signals.
  - After execution of each instruction, the end signal is generated which resets control step counter and makes it ready for generation of control step for next instruction.

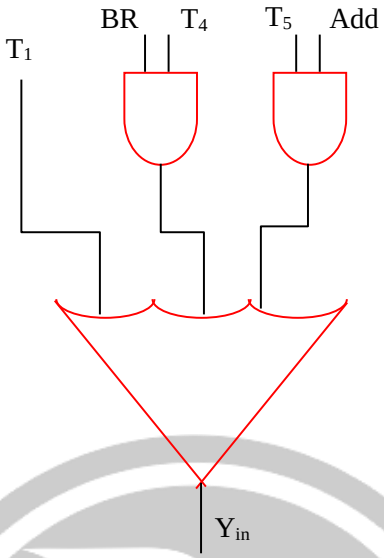
For example:

(1) The encoder circuit implements the following logic function to generate Y_{in}.

$Y_{in} = T_1 + T_5$ Add $+ T_4$ BR $+ \ldots..$

i.e. Here Y_{in} signal is asserted during time interval T_1 for all instructions, during T_5 for add instruction, during T_4 for branch instruction.

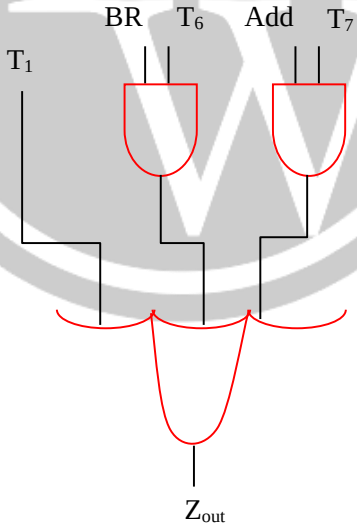
The generation of Y_{in} control signal is



(2) The logic function to generate Z_{out} signal can be given by

$Z_{out} = T_2 + T_7$ add + T_6 BR + ……

i.e., the Z_{out} signal is asserted during time interval T_2 for all instructions, during T_7 for add instruction, during T_6 for unconditional branch …….. etc.

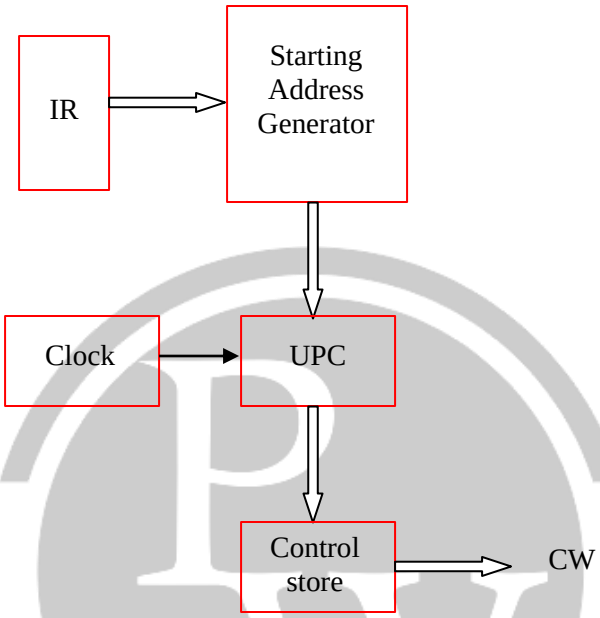


4.2 Microprogrammed Control

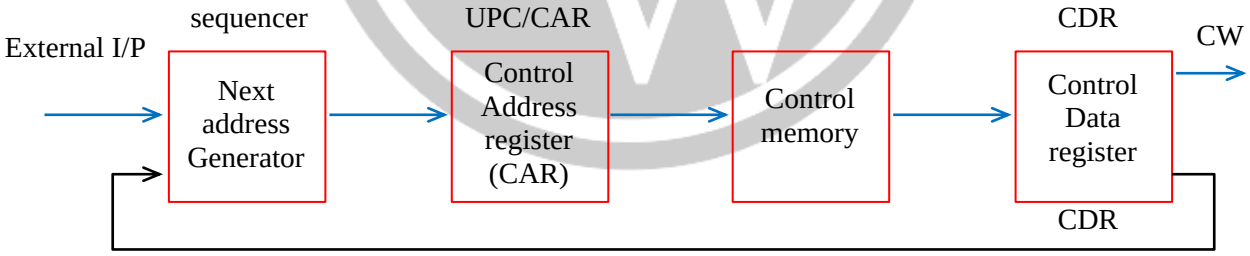
- Every instruction in a processor is implemented by a sequence of one or more sets of micro operations. Each micro operation is associated with a specific set of control lines which when activated, causes that micro operation to take place.
- Using micro programmed control, control signals are generated by a program. Using this the control signal selection and sequencing information is stored in ROM or RAM called control memory (CM).

- The control memory is part of control unit; it contains fixed programs (micro programs) that cannot be altered by occasional user.
- A control word (CW) is a word whose individual bits represent the various control signals.
- A sequence of CWs corresponding to the control sequence of a machine instruction constitutes the micro routine for that instruction, (micro program)
- The individual control words in the microprogram are referred as micro instruction.

- The basic organization of a microprogrammed control unit is



- To read the control words sequentially from the control memory a "micro program counter" (μPC) is used.



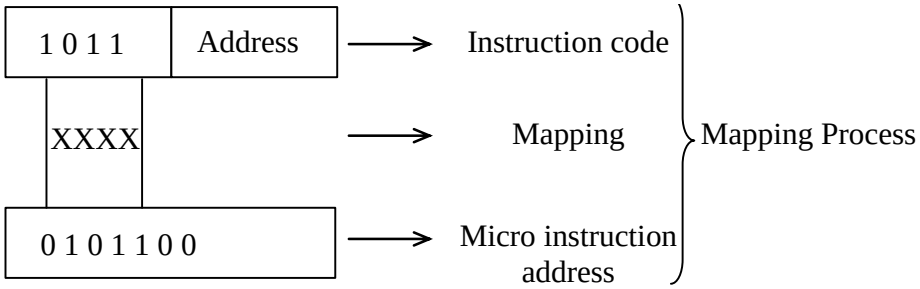
- The sequence of operations are:
  (1) CAR holds the address of next micro instruction to be read.
  (2) When address is available in UPC, the sequencer issues the READ command to CM.
  (3) The word from addressed location is read into CDR/UIR.
  (4) The UPC is incremented automatically by the clock, causing successive micro instructions to be read from CM.
  (5) The contents of UIR generates control signals which are delivered to various parts of the processor in correct sequence.
- The μPC or CAR can be updated with various options using the address sequencer circuit as:

(1) When a new instruction is loaded in IR, the μPC is loaded with the starting address of the micro routine for that instruction (called mapping process).

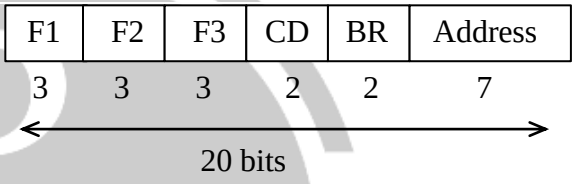
(2) When a branch instruction is (micro) encountered and branch condition satisfied, the μPC is loaded with the branch address.

(3) When END instruction is encountered, the μPC is loaded with address of first word.

(4) In any other situation, the μPC is incremented every time a new micro instruction is fetched from CM.



- The transformation from the instruction code bits to an address in control memory where the routine is located is called Mapping process.
- The basic microinstruction format is
- The function fields (F1, F2, F3,), condition field (CD) & Branch field (BR) may be optional.



Comparison between Hardwired & Microprogramed

	Hardwired	**Micro programmed**
1. Speed	Fast	Slow
2. Control functions	Implemented in Hardwire	Implemented in software
3. Ability to handle complex instructions	Complex	Easier
4. Design process	Complicated	Orderly and systematic
5. Instruction set size	Under 100	Over 100
6. ROM size	NIL	2k to 10k (20–400 bit micro instruction)
7. Applications	RISC processors	CISC, Main frames

4.2.1. The Micro Programmed Control Unit Can be

(1) Horizontal Microprogramming:

- One bit per control signal.
- Maximum parallelism i.e., more than one signal can be simultaneously.
- No extra decoders are required for decoding.
- The length of control word is large, need to access more than once from control memory.

(2) Vertical Microprogramming:

- The control signals are encoded in form for k-bits 2^k signals.
- Maximum degree of parallelism is 1.
- The length of micro instruction is small.
- Response is relatively slower.
- Requires a decoder additionally.

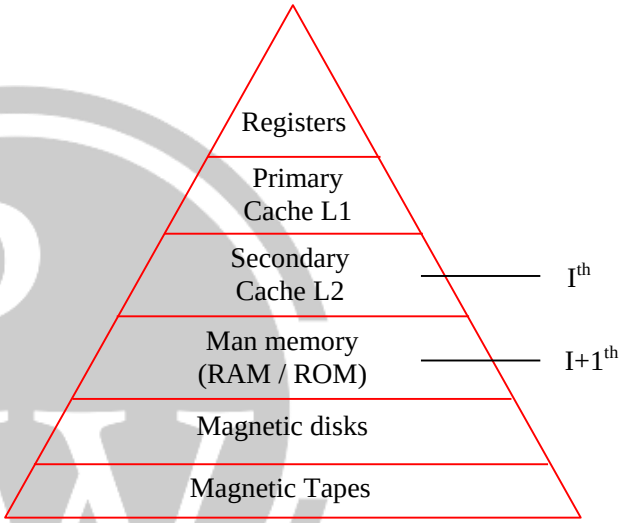
- To increase the degree of parallelism soft vertical microprogramming is used which divides the control signals into mutually exclusive groups. Each group is associated with associated number of control bits. (i.e., combination of both vertical and horizontal microprogramming).

❑❑❑

5 MEMORY INTERFACING

5.1 Memory Hierarchy

- The memory hierarchy system consists of all storage devices.
  The purpose of memory hierarchy is to bridge the speed mismatch between the fastest processor to the slowest memory component at reasonable cost.



- In the structure of memory hierarchy I^{th} level memory is physically positioned higher than $(I + 1)^H$ level memory.
- Let Ti, Si, Ci, & Fi are the access time, size, cost per bit and frequency of references to the Ith level memory. Therefore

$$Ti < Ti + 1$$

$$Si < Si + 1$$

$$Ci > Ci + 1$$

$$fi > fi + 1$$

- Since same data presents at different levels. I^{th} level memory is the subset of I i + 1th level.

  i.e. $I\,i \subset I\,i + 1$

5.2 Memory Characteristics

Location: Memory can be placed in 3 locations like registers, main memory, Auxiliary (or) secondary storage.

Capacity: Word size, number of words i.e. capacity = number of words * word size.

Unit of transfer: Maximum number of bits that can be read or written (blocks, bytes…)

Access method: Random or sequential

Performance: Access time, memory cycle time, transfer rate, physical type.

Physical: Volatile / non volatile

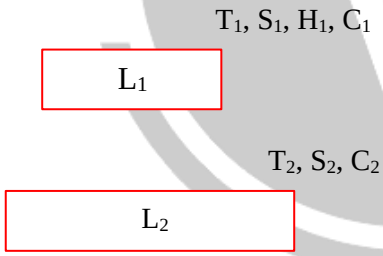
erasable / non erasable

	Serial Access		Random Access
(1)	Memory is organized into units of data called records/blocks accessed sequentially	**(1)**	Each storage location has an address uniquely
(2)	Access time depends on position of storage location	**(2)**	Access time is independent of storage location
(3)	Slower to Access	**(3)**	Faster to access
(4)	Cheaper	**(4)**	Costly relatively
(5)	Nonvolatile memories	**(5)**	May be relative or non
(6)	Example: Magnetic tapes	**(6)**	Example: Magnetic disks.

- When processor reads I^{th} level memory, if it is found in that level, his will occur otherwise it will be fault.

5.2.1. In a Two – Level Memory System

Case – I: When fault occurs, in L1 reads from L2 (Proxy Hierarchy)



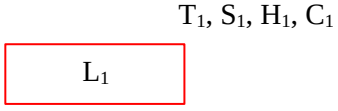
$T_{avg} = H_1 * T_1 + (1–H_1) * T_2$

Case – II: When fault occurs in L_1, must be brought from L2 to L1 memory. (**strict hierarchy**)

$T_{avg} = H_1 * T_1 + (1–H_1) * (T_2 + T_1)$

5.2.2. In a Three – Level Memory System

Case – I: When fault occurs in one level, then reads from its down level.



T_2, S_2, H_2, C_2

L_2

T_2, S_2, H_2, C_2

L_3

$T_{avg} = H_1 * T_1 + (1 – H_1) * H_2 * T_2 + (1 – H_1) * (1 – H_2) * T_3$

Case – II: When fault occurs, must access from L1.

$T_{avg} = H_1 * T_1 + (1–H_1)\,(H_2)\,(T_2 + T_1) + (1–H_1)\,(1–H_2)\,(T_3 + T_2 + T_1)$

* Average cost per bit

$$C_{Avg} = \frac{C_1S_1 + C_2S_2}{S_1 + S_2} \quad \text{(Two - level)}$$

$$C_{Avg} = \frac{C_1S_1 + C_2S_2 + C_3S_3}{S_1 + S_2 + S_3} \quad \text{(Three - level)}$$

❑❑❑

6 INPUT OUTPUT INTERFACING

6.1 Input – Output Organization

- The input – output subsystem (I/O) provides an efficient mode of communication b/n system and outside environment. The Commonly used peripheral devices are. Keyboard, monitors, printer & magnetic tape, magnetic disc…..
- The input & output devices that communicates with computer & people usually with alphanumeric character from ASCII – 128-character set, 7 – bits are used represent each character. It contains 26 upper case letters, 26 lower case letters, 10 numerals, 32 special chars such as %, *, $ …… In general ASCII chars are stored in a single unit called 1 byte (8-bits). The 8th bit may be used for parity bit for error detection.

6.1.1. Input – Output Interface

- The I/o interface provides a method of transferring information b/n internal storage (main memory) & external I/o devices. The devices/ peripherals connected to a computer need special communication links for interfacing with CPU because.
  - Peripherals are electromechanical & electromagnetic devices, whereas CPU & memory are electronic devices hence the operations are different.
  - The data transfer rate is shown then the transfer rate of CPU.
  - The data codes & formats are different.
  - The operating modes of peripherals is different and must be controlled.
- The four types of I/o command an interface will receive are

6.1.2. Control Command

Issued to activate the peripheral & and to inform it what to do. Depending on mode of operation a control command sequence is issued.

6.1.3. Status Command

Used to test various status conditions in the interface & the peripheral. Eg. Checking status of device, errors detected by interface. The status information is maintained in status register.

6.1.4. Data Output

Causes the interface to respond by transferring data from the bus into one of its registers buffer.

6.1.5. Data Input

With this command, interface receives an item of data from peripheral & places it in its buffer register. Then transfers to data bus of processor.

The I/o read & I/o write are enabled during I/o transfer, memory ready & memory write are enabled during memory transfer. The two configurations possible for communication are.

(a) Isolated I/o:

The CPU has distinct input & output instructions, each instruction is associated with the address of an interface register. When CPU fetches & decodes the opcode, it places address associated with the instruction into the common address lines, at the same time, it enables the I/o read or I/o write control line.

An isolated I/o method isolates memory & I/o addresses each has its own address space, hence memory address values are not affected by interface address. Uses one common bus for memory & I/o, with common control lines.

(b) Memory – Mapped I/o:

In this configuration, only one set of read & write signals and do not distinguish b/n memory & I/o addresses. and I/o.

6.2 Modes of Data Transfer

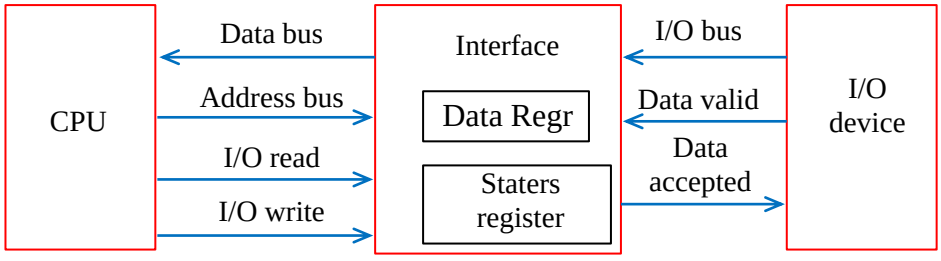
Data transfer b/n the central computer (main memory) and I/o device may be handled in a variety of modes like programmed I/o, Interrupt – Initiated I/o & Direct memory access.

6.2.1. Programmed I/O

- Programmed I/o operations are the result of I/o instruction written in computer program:

Example:

In programmed I/o, the I/o device doesn't have direct access to memory. A transfer from I/o device to memory requires the CPU to extents several instruction.



F – flag bit

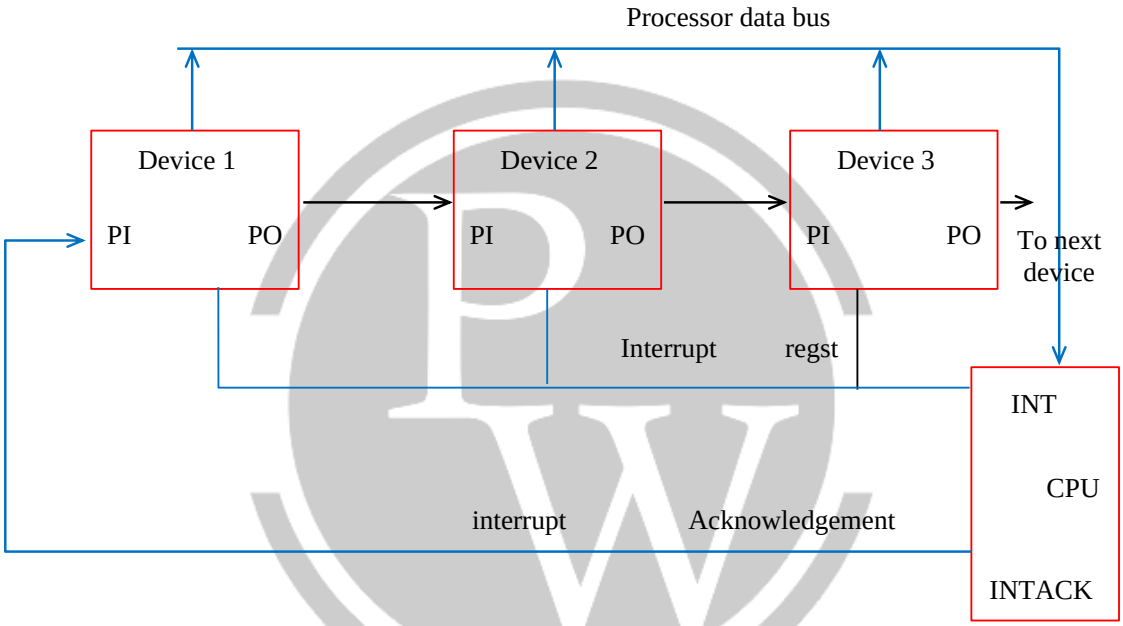
- (Data transfer from I/o device to CPU) The device transfers bytes of data one at a time as they are available. When a byte of data is available the device places it on I/o bus and enables its data valid line. The Interface accepts the byte into its data register and enables the data accepted line. The interface sets the Flag bit. Then the device disables the data valid line, but it will not transfer another byte until the data accepted line is disabled by the interface.

6.2.2. Interrupt – Initiated I/O

- In programmed I/O, the CPU stays in a program loop until the I/o unit indicates that it is ready to transfer. Hence this process keeps processor busy needlessly. meanwhile keeps monitoring the device. When the interface determining that the device is ready, it generates an interrupt request
- **Priority Interrupt:** It is a system that establishes a priority over the various sources to determine which condition is to be serviced first when two or more requests arrive simultaneously. It also determines which conditions are permitted during processing of an

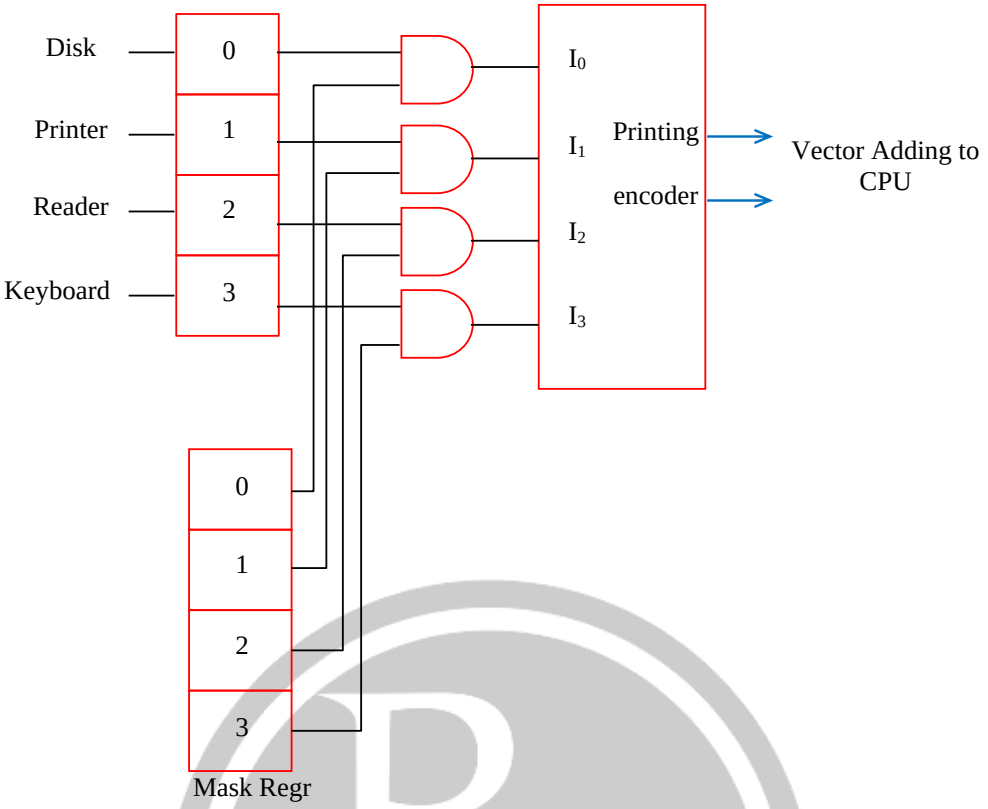
6.2.3. Daisy – Chaining Priority (Serial – Priority Interrupt)

The system consists of a serial connection of all devices that request an interrupt. The device with the highest priority is placed in first position followed by lower – priority devices. The lowest priority will be placed last in the chain.



6.2.4. Parallel Priority Interrupt

- This method uses a register whose bits are set separately by the interrupt signal from each device Priority is established according to the position of bits in the register. The CKT will also include a mask register to control the starting of each interrupt request. The mask register can be programmed to disable low-priority interrupts while a high – priority device is being revised. If will also allow a high-priority device to interrupt the CPU while low – priority device is being serviced.



6.2.5. Direct Memory Access (DMA)

- The speed of data transfer can be increased by removing CPU from the path and the peripheral device to manage the memory buses directly. This kind of transfer technique is called DMA transfer during DMA transfer the CPU is idle and has no control of the memory buses. The DMA controller takes control of buses to manage the transfer directly b/n i/o device & memory.
- To keep CPU idle to use bus, the "bus request (BR) input is used by the DMA controller to request the bus to relinquish control of the buses. When BR is active, CPU terminates current execution and places data, control & address lines in high impedance state. Then the bus behaves like an open circuit. The CPU then activates. "Bus grant" (BG) output to DMA, then DMA takes control of the bus to transfer without CPU intervention, when the transfer completes DMA disables the Bus request & CPU disables the bus grant. Then the CPU gets the control of the buses.

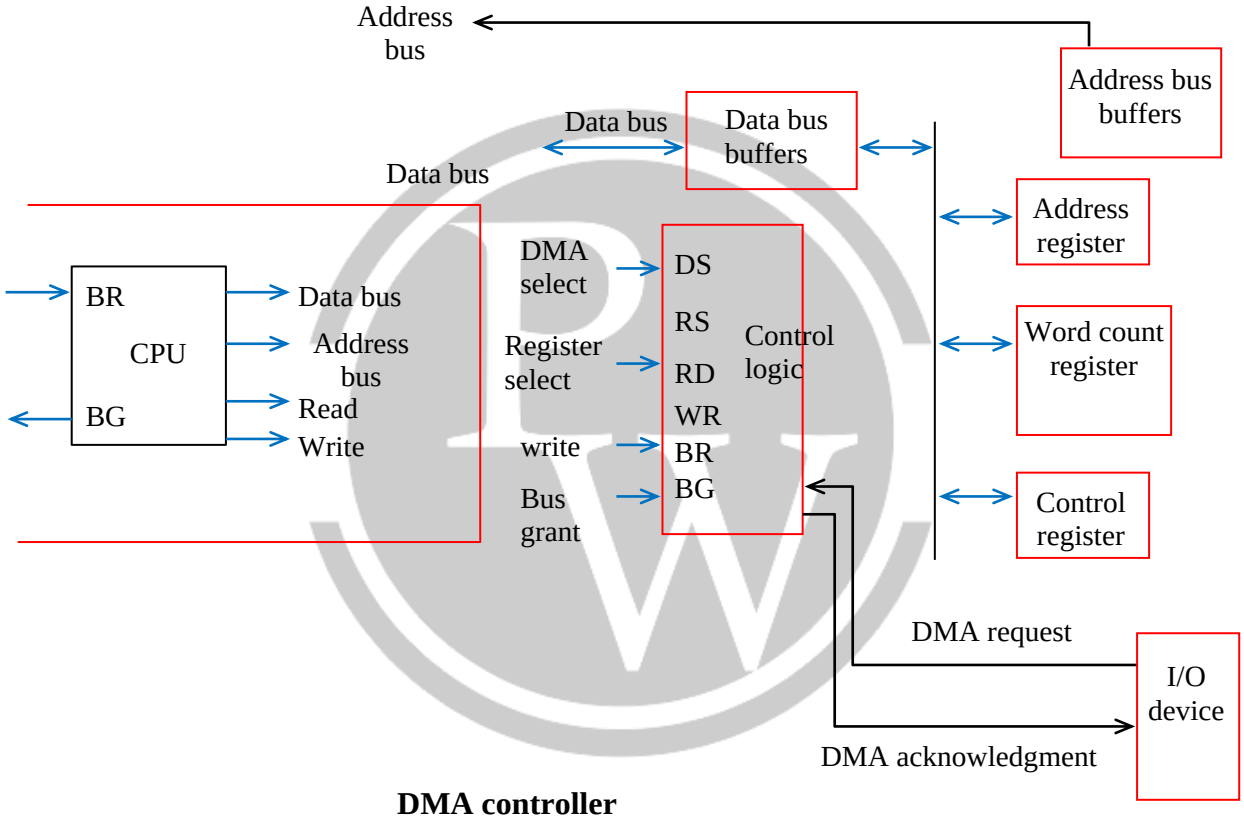
6.3. The DMA Transfer Take Place in Two Modes

(1) Burst transfer:

A block sequence consisting of a number of memory words is transferred in a continuous burst while DMA controller is master of the memory buses. Used to transfer fast devices like magnetic disks for large volumes. In this mode the data transfer will not be stopped, until an entire block is transferred.

(2) Cycle Stealing:

In this mode, DMA controller transfers one word at a time, after which it must return control of the buses to the CPU, later it will 'steal' memory cycle when CPU is idle.



DMA controller

- The DMA controller communicates with the CPU the data bus and control lines. The register in DMA are selected by CPU through address bus by enabling DS & RS inputs. When BG = 1 (bus grant) the CPU has relinquished the buses and the DMA can communicate directly with the memory by specifying address in address bus & activating the RD or WR control. The DMA communicates with the external device through the request and acknowledge lines.

6.4. The CPU Initialize the DMA by Sending

(1) The starting address of memory block where data are available (for read) or where data are to be stored (for write)

(2) Word count, the no of words in memory block.

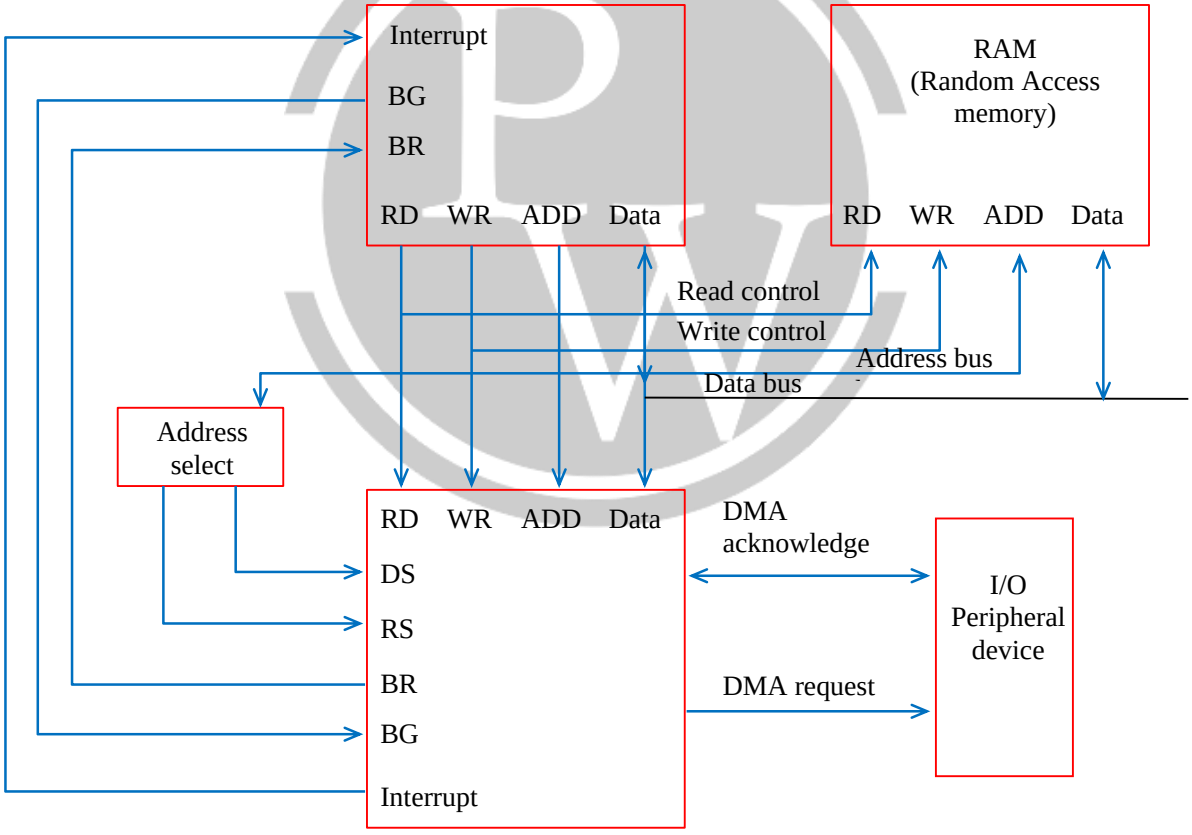
(3) Control of specify mode of transfer such as read or write.

(4) A control to start the DMA transfer.

- Once DMA is initialized, the CPU stops communicating with the DMA unless it receivers an interrupt signal.

6.5. DMA Transfer

When the peripheral device sends a DMA request, the DMA controller activates the BR line, informing the CPU to relinquish the buses. The CPU responds with its BG line, informing DMA that its buses are disabled. DMA then puts the current value of its address register into the adder bus and initiates RD or WR signal. It then sends DMA acknowledge to the peripheral device. When BG = 0, then CPU communicates with the internal DMA registers, when BG = 1 then RD & WR lines are used from DMA controller to RAM to specify read or write operation.



When the device receives DMA acknowledge, it puts a word in the data bus (write) or receives on word from the data bus (read). In this way the peripheral communicates with memory without any involvement of CPU. For each word transfer DMA increments the address register and decrements its word count register. When count reaches to zero, the DMA disables bus request line so that the CPU can continue to execute its program DMA transfer is very useful for fast transfer of data.

❑❑❑

7 PIPELINING

7.1 Introduction

- "Parallel processing" denotes a large class of techniques that are used to provide simultaneous data–processing tasks for the purpose of increasing the computational speed of a computer system.
- The purpose of parallel processing is to speed up the computer processing capability and increases its throughput. "Through put" is the amount of processing that can be accomplished during a given interval of time".
- Parallel processing can be viewed from various levels as

  (i) At registers level, Registers with parallel load operate with all the bits of the word simultaneously. (Instead of shift registers). This is at lowest level.

  (ii) Higher level of complexity can be achieved by having a multiplicity of functional units that perform identical or different operations simultaneously.

  (iii) By distributing the data among the functional units in multiple. Eg: The arithmetic logical & shift operations can be separated.

  (iv) Using multiple processors. (Flynn's classification) (SISD, SIMD, MISD, MIMD).

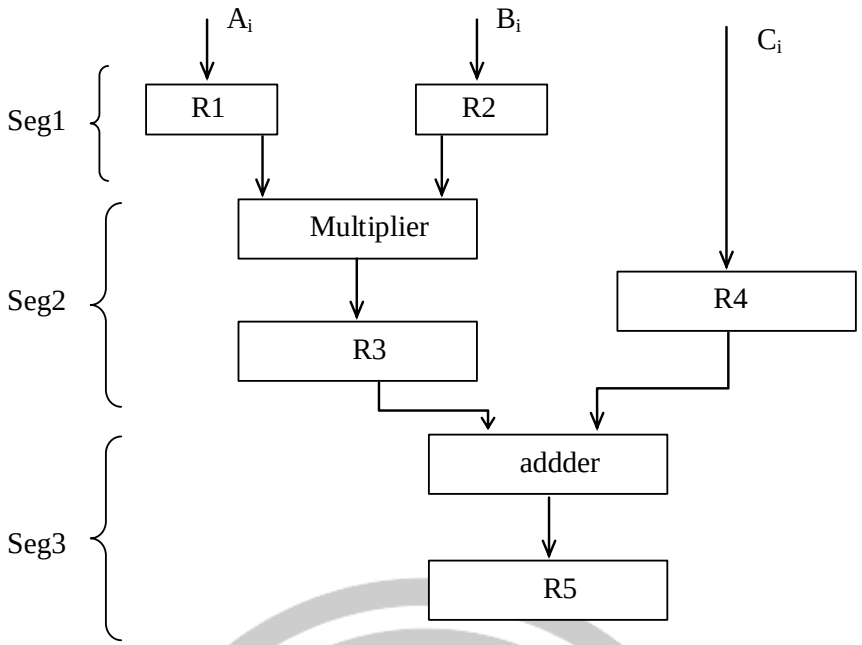
  (v) Using pipelining in unit processor systems.

7.2 Pipelining

- Pipelining is a technique of decomposing a sequential process into sub operations, with each sub process being executed in a special dedicated segment that operates concurrently with all other segments.
- The result obtained from one segment is transferred to the next segment in the pipeline. The final result is obtained after the data have passed through all segments.

Example:

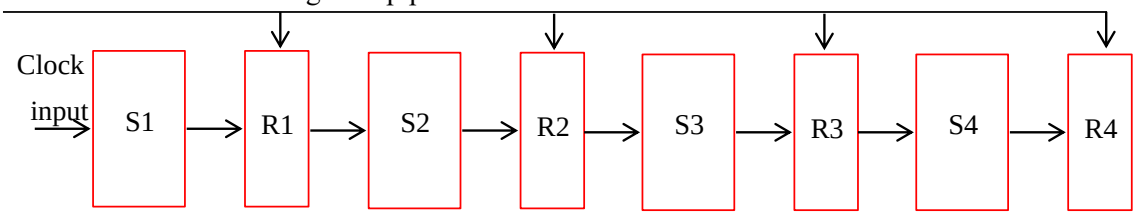
The perform Ai * Bi + Ci , for i = 1 to 6. Each sub operation multiply & add implemented in a segment with in a pipeline. Each segment has one or more registers.



- R1 through R5 are registers that receive new data with every clock pulse.
- The effect of each clock pulse can be shown as

Clock pulse Number	Segment – 1		Segment - 2		Segment - 3
	R1	R2	R3	R4	R5
1	A1	B1	---	---	---
2	A2	B2	$A_1 * B_1$	C1	---
3	A3	B3	A2 * B2	C2	A1 * B1 + C1
4	A4	B4	A3 * B3	C3	A2 * B2 + C2
5	A5	B5	A4 * B4	C4	A3 * B3 + C3
6	A6	B6	A5 * B5	C5	A4 * B4 + C4
7	---	---	A6 * B6	C6	A5 * B5 + C5
8	---	---	---	---	A6 * B6 + C6

- The "General structure" of a "4 – segment pipeline" is



- The operands pass through all four segments in a fixed sequence.
- Each segment consists of combinational circuit Si that performs a sub operation.
- The segments are separated by registers Ri that holds the intermediate results between stages.
- Information flows between adjacent stages under the control of common clock applied to all registers simultaneously.
- A task as the total operation performed going through all the segments in the pipeline.
- The behaviour of a pipeline can be illustrated with "Space – time diagram". It shows the segment utilization as a function of time. The following diagram is for tasks T1 to T6 executed in 4 – segment pipeline.

Segments	1	2	3	4	5	6	7	8	9
S – 1	T1	T2	T3	T4	T5	T6			
S – 2		T1	T2	T3	T4	T5	T6		
S – 3			T1	T2	T3	T4	T5	T6	
S – 4				T1	T2	T3	T4	T5	T6

Clock cycles

- "An Arithmetic pipeline divides an arithmetic operation into sub operations for execution in the pipeline segments."

7.2.1 Pipeline performance evaluation

- Consider a K – segment pipeline with a clock cycle time t_p is used to execute n tasks.
- To complete n – tasks using a K – segment pipeline requires K + (n–1) clock cycles.

(1) The number of clock cycles needed in a pipeline to execute 100 tasks in 6 segments.

Ans. K = 6

n = 100

$\Rightarrow$ 6 + (100-1) = 105 clocks

- The processing time in each stage is called as "stage delay", In a pipeline. (t_p).
- The time delay due to interstage transfer of data is "interstage delay" (td)
- The interstage delays can be same between the stages, stage delay very from stage to stage based on segment operation.

The time period for clock cycle, 't'$_p$ is

$t_p = \max\{t_i\} + t_d$

$t_p = t_m + t_d$

Since $t_m >> t_d$, maximum stage delay denotes the clock period.

i.e. $t_p = t_m$

- The total time required in pipeline execution is

$T_P = [K + (n–1)] * t_p$.

7.2.2 Speed up Ratio

- The speed up of a K – stage pipeline over an equivalent non-pipelined processor is defined as

$$S = \frac{\text{time without pipeline}}{\text{time with pipeline}}$$

- Consider a non-pipeline processor that performs the same operation as pipelined and takes a time equal to "t_n" to complete each task.

$$S = \frac{n * t_n}{(K + (n-1)) * t_p}$$

- As the number of tasks increases, k + n - 1 approaches to n

Since $t_n \simeq K * t_p$

$$S = \frac{n * K * t_p}{(K + (n-1)) * t_p} \qquad \therefore t_n \simeq nt_p$$

$$S = \frac{nK}{K + (n-1)}$$

$$S = \frac{n\,t_n}{n * t_p} \qquad \therefore K + n - 1 \simeq n$$

$$\boxed{S = \frac{t_n}{t_p}}$$

For large number of tasks

$$S = \frac{nk}{K + n - 1} \qquad S = \frac{t_n}{t_p}$$

Or

$$= \frac{nK}{n} \qquad S = \frac{K \times t_p}{t_p}$$

$$\boxed{S = K} \qquad \boxed{S = K}$$

- The maximum speed up that can be achieved in a pipelined processor is equal to "number of stages". [as n is large, K + n –1 is K]. (K).

$$S_{ideal} = S_{max} = K$$

7.2.3 Efficiency

- The efficiency of a pipeline is defined as the ratio of speed up factor and the number of stages in the pipeline.

$$E_k = \frac{S}{K} = \left(\frac{nk}{K + (n-1)}\right) / K$$

$$E_K = \frac{n}{K+n-1}$$

$$E_K = \frac{S}{K}$$

7.2.4 Throughput

- It is the number of tasks that can be completed by a pipeline per unit time.

$$H_K = \frac{n}{(K+(n-1))\times t_p}$$

7.2.5 Stall cycle

- The performance of a pipeline is influence by

  Number of instructions

  Uneven stage delays

  Buffer overhead (Interstage delay)

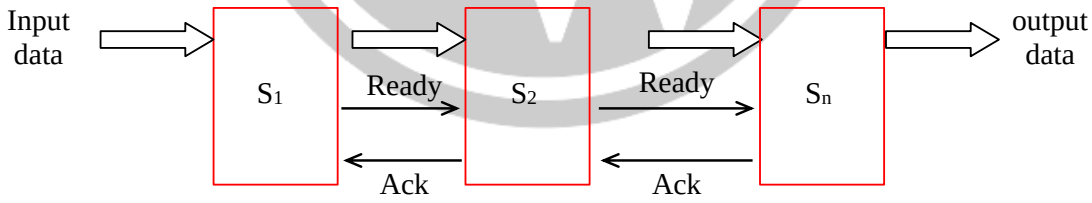
  Dependencies.
- The objective of pipelines is $CPI_{avg} = 1$.

  (i.e. Clocks per instruction – CPI)
- Depending on control mechanism used, the pipelines can be categorized as

7.2.6 I Asynchronous pipeline

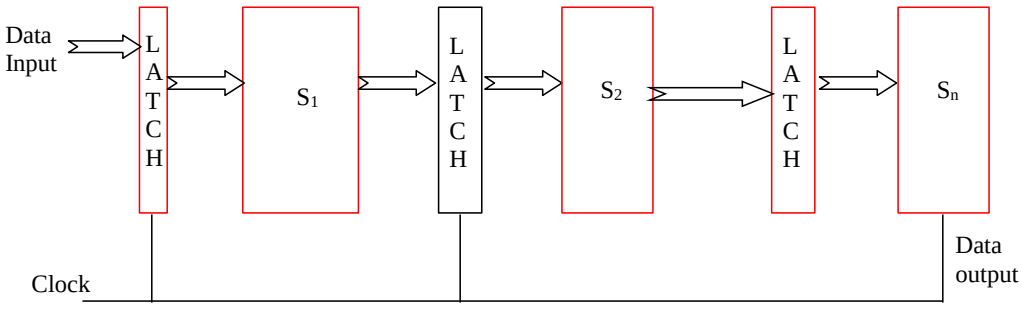
- Data flow along the pipeline stages is controlled by handshaking protector i.e. when S_i is ready to send/transmit, it sends ready signal to S_{i+1} and S_{i+1} send A.

  ck to Si after receiving.



7.2.7 II Synchronous pipeline

- Clocked high speed latches are used to interface between stages. At the falling edge of the clock pulse, all latches transfer data to the next stages simultaneously.



7.2.8 Instruction pipelining

- An instruction pipeline operates on a stream of instructions by overlapping the Fetch, Decode, Execute and other phases of instruction cycle.
- The instruction cycle with 4 – segments is

Instructions	1	2	3	4	5	6	7	8	9	Clock
I 1	FI	DA	FO	FX						
I 2		FI	DA	FO	EX					
I 3			FI	DA	FO	EX				
I 4				FI	DA	FO	EX			
I 5					FI	DA	FO	EX		

Here FI - fetches an instruction

DA - Decodes the instruction & calculates effective address

FO - fetches the operand

EX - Executes the instruction

- "In general, each stage in a pipeline is expected to complete its operation in one clock cycle, hence the clock period must allow the longest task to be completed.
- "The performance of a pipeline is high if different stages require about same amount of time."
- The use of cache solves the memory access problem.

(5) Consider a 4–stage pipeline, where different instructions require different number of clock cycles, at different stages. The total number of clocks required for execution of 4 instructions is _____

Ans.:

	1	2	3	4	
I 1	2	1	2	2	**7**
I 2	1	3	3	2	**9**
I 3	2	2	2	2	**8**
I 4	1	3	1	1	**6**
					30

Through sequential process 30 clocks. But using pipeline the number of clock cycles required is 14. That is,

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**
I1	S1	S1	S2	S3	S3	S4	S4							
I2			S1	S2	S2	S2	S3	S3	S3	S4	S4			
I3				S1	S1	-	S2	S2	-	S3	S3	S4	S4	
I4						S1	-	-	S2	S2	S2	S3	-	S4

7.2.9 Data Hazards

- A data hazard is situation in which the pipeline is stalled because the data to be operated on are delayed for some reason, as illustrated in Figure. We will now examine the issue of availability of data in some detail.
- Consider a program that contains two instructions, I_1 followed by I_2. When this program is executed in a pipeline, the execution of I_2 can begin before the execution of I_1 is completed. This means that the results generated by I_1 may not be available for use by I_2. We must ensure that the results obtained when instructions are executed in a pipelined processor are identical to those obtained when the same instructions are executed sequentially. The potential for obtaining incorrect results when operations are performed concurrently can be demonstrated by a simple example. Assume that A = 5, and consider the following two operations:

$$A \leftarrow 3 + A$$

$$B \leftarrow 4 \times A$$

- When these operations are performed in the order given, the result is B = 32. But if they are performed concurrently, the value of A used in computing B would be the original value, 5, leading to an incorrect result. If these two operations are performed by instructions in a program, then the instructions must be executed one after the other, because the data used in the second instruction depend on the result of the first instruction. On the other hand, the two operations.

$$A \leftarrow 5 \times C$$

$$B \leftarrow 20 + C$$

- Can be performed concurrently, because these operations are independent.

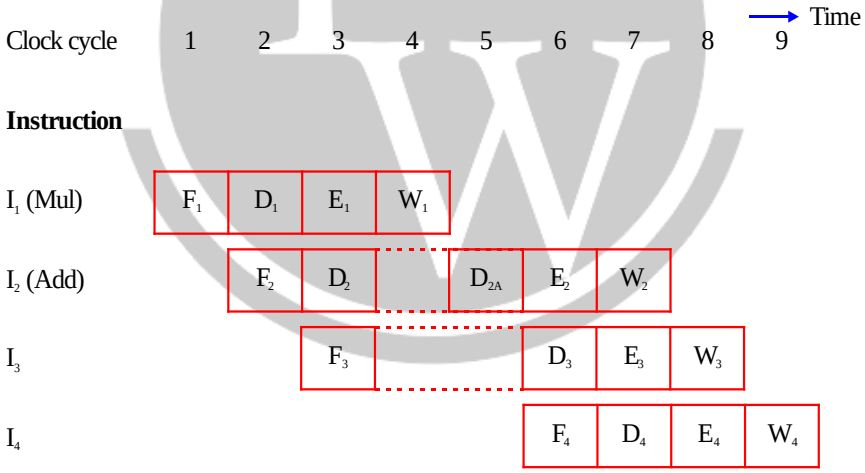


Fig: Pipeline stalled by data dependency between D_2 and W_1

- This example illustrates a basic constraint that must be enforced to guarantee correct results. When two operations depend on each other, they must be performed sequentially in the correct order. This rather obvious condition has far-reaching consequences. Understanding its implications is the key to understanding the variety of design alternatives and trade-offs encountered in pipelined computers.
- Consider the pipeline in Figure. The data dependency just described arises when the destination of one instruction is used as a source in the next instruction. For example, the two instructions

Mul R2.R3.R4

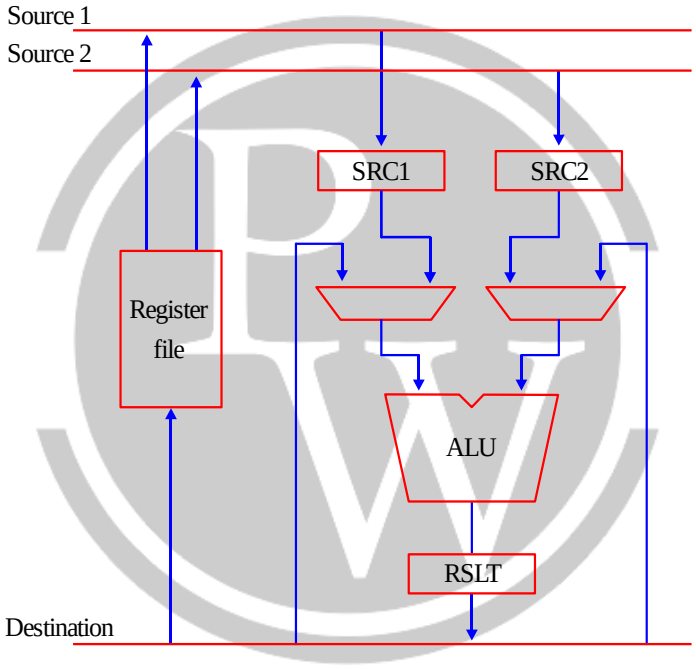
Mul R5.R4.R6

- Give rise to a data dependency. The result of the multiply instruction is placed into register R4. which in turn is one of the two source operands of the Add instruction. Assuming that the multiply operation takes one clock cycle to complete; execution would proceed as shown in Figure. As the Decode unit decodes the Add instruction in cycle 3, it realizes that R4 is used as a source operand. Hence, the D step of that instruction cannot be completed unit the W step of multiply instruction has been completed. Completion of step D_2 must be delayed to clock cycle 5, and is shown as step D_{2A} in figure. Instruction I_3 is fetched in cycle 3, but its decoding must be delayed because step D_3 cannot precede D_2. Hence, pipelined execution is stalled for two cycles.

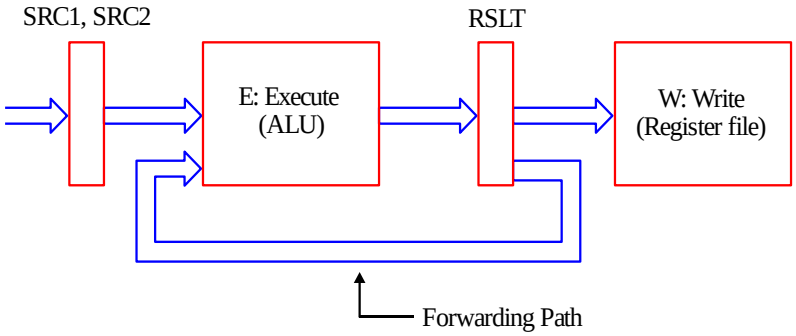
7.2.10 Operand Forwarding

- The data hazard just described arises because one instruction, instruction I_2 in Figure, is waiting for data to be written in the register file. However, these data are available at the output of the ALU once the Execute stage completes step E_1. Hence, the delay can be reduced, or possibly eliminated, if we arrange for the result of instruction I_1 to be forwarded directly for use in step E_2.

Figure shows a part of the processor data path involving the ALU and the register file.



(a) Datapath



(b) Position of the source and result registers in the processor pipeline

- Interstage buffers needed for pipelined operation, as illustrated in Figure. With reference to Figure, registers SRC1 and SRC2 are part of buffer B2 and RSLT is part of B3. The data forwarding mechanism is provided by the blue connection

lines. The two multiplexes connected at the inputs to the ALU allow the data on the destination bust to be selected instead of the contents of either the SRC1 or SRC2 register.

- When the instructions in Figure 8.6 are executed in the data path of Figure, the operations performed in each clock cycle are as follows. After decoding instruction I_2 and detecting the data dependency, a decision is made to use data forwarding. The operand not involved in the dependency, register R2, is read and loaded in register SRC1 in clock cycle 3. In the next clock cycle, the product produced by instruction I_1 is available in register RSLT, and because of the forwarding connection, it can be used in step E_2. Hence, execution of I_2 proceeds without interruption.

7.2.11 Handling Data Hazards in Software

- In Figure, we assumed the data dependency is discovered by the hardware while the instruction is being decoded. The control hardware delays reading register R4 unit cycle 5, thus introducing a 2-cycle stall unless operand forwarding is used. An alternative approach is to leave the task of detecting data dependence and dealing with them to the software. In this case, the compiler can introduce the two-cycle delay needed between instructions I_1 and I_2 by inserting NOP (No-operation) instructions, as follows:

I_1 : Mul R2, R3, R4
NOP
NOP
I_2 : Add R5, R4, R6

- If the responsibility for detecting such dependencies is left entirely to the software, the compiler must insert the NOP instructions to obtain a correct result. This possibility illustrates the close link between the compiler and the hardware. A particular feature can be either implemented in hardware or left to the compiler. Leaving tasks such as inserting NOP instructions to the compiler leads to simpler hardware. Being aware of the need for a delay, the compiler can attempt to reorder instructions to perform useful tasks in the NOP slots, and thus achieve better performance. On the other hand, the insertion of NOP instructions leads to larger code size. Also, it is often the case that a given processor architecture has several hardware implementations, offering different features.

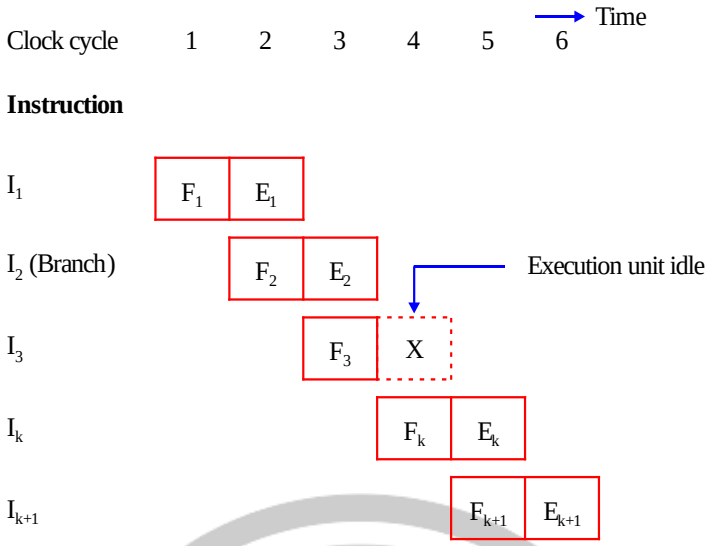
7.3 INSTRUCTION HAZARDS

- The purpose of the instruction fetch unit is to supply the execution units with a steady stream of instructions. Whenever this stream is interrupted, the pipeline stalls, as Figure illustrates for the case of cache miss. A branch instruction may also cause the pipeline to stall. We will now examine the effect of branch instructions and the techniques that can be used for mitigating their impact. We start with unconditional branches.

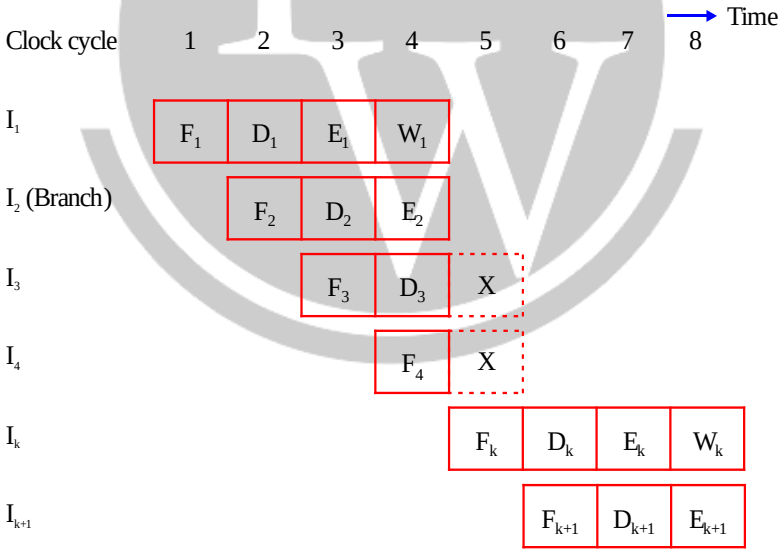
7.4 UNCONDITIONAL BRANCHES

- Figure shows a sequence of instructions being executed in a two-stage pipeline. Instructions I_1 to I_3 are stored at successive memory addresses, and I_2 is a branch instruction. Let the branch target be instruction I_h. In clock cycle 3, the fetch operation for instruction I_3 is in progress at the same time that the branch instruction is being decoded and the target address computed. In clock cycle 4, the processor must discard I_3, which has been incorrectly fetched, and fetch instruction I_h. In the meantime, the hardware unit responsible for the Execute (E) step must be told to do nothing during that clock period. Thus, the pipeline is stalled for one clock cycle.
- The time lost as a result of a branch instruction is often referred to as the *branch penalty*. In Figure, the branch penalty is one clock cycle. For a longer pipeline, the branch penalty may be higher. For example, Figure shows the effect of a branch

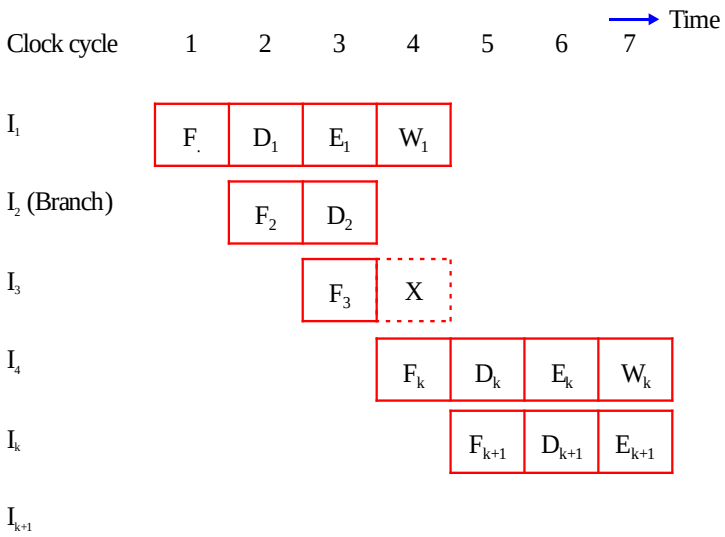
instruction on a four-stage pipeline. We have assumed that the branch address is computed in step E_2. Instructions I_3 and I_4 must be discarded, and the target instruction, I_h, is fetched in clock cycle 5. Thus, the branch penalty is two clock cycles.



- Reducing the branch penalty requires the branch address to be computed earlier in the pipeline. Typically, the instruction fetch unit has dedicated hardware to identify a branch instruction and compute the branch target address as quickly as possible after an instruction is fetched. With this additional hardware, both of these tasks can be performed in step D_2, leading to the sequence of events shown in Figure. In this case, the branch penalty is only one clock cycle.



(a) Branch address computed in Execute stage



(b) Branch address computed in Decode stage

7.4.1 Instruction Queue and Prefetching

- Either a cache miss or a branch instruction stalls the pipeline for one or more clock cycles. To reduce the effect of these interruptions, many processors employ sophisticated fetch units that can fetch instructions before they are needed and put them in a queue. Typically, the instruction queue can store several instructions. A separate unit, which we call the *dispatch unit*, takes instructions from the front of the queue and

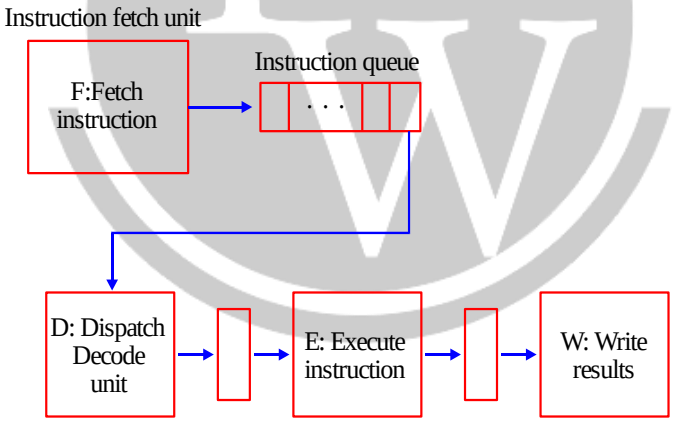


Fig: Use of an instruction queue in the hardware organization of fig (b)

- Sends them to the execution unit. This leads to the organization shown in Figure 8.10. The dispatch unit also performs the decoding function.
- To be effective, the fetch unit must have sufficient decoding and processing capability to recognize and execute branch instructions. It attempts to keep the instruction queue filled at all times to reduce the impact of occasional delays when fetching instructions. When the pipeline stalls because of a data hazard, for example, the dispatch unit is not able to issue instructions from the instruction queue. However, the fetch unit continues to fetch instructions and add them to the queue. Conversely, if there is a delay in fetching instructions because of a branch or a cache miss, the dispatch unit continues to issue instructions from the instruction queue.

7.5 CONDITIONAL BRANCHES AND BRANCH PREDICTION

- A conditional branch instruction introduces the added hazard caused by the dependency of the branch condition on the result of a preceding instruction. The decision to branch cannot be made until the execution of that instruction has been completed.
- Branch instructions occur frequently. In fact, they represent about 20 percent of the dynamic instruction count of most programs. (The dynamic count is the number of instruction executions, taking into account the fact that some program instructions are executed many times because of loops.) Because of the branch penalty, this large percentage would reduce the gain in performance expected from pipelining. Fortunately, branch instructions can be handled in several ways to reduce their negative impact on the rate of execution of instructions.

7.5.1 Delayed Branch

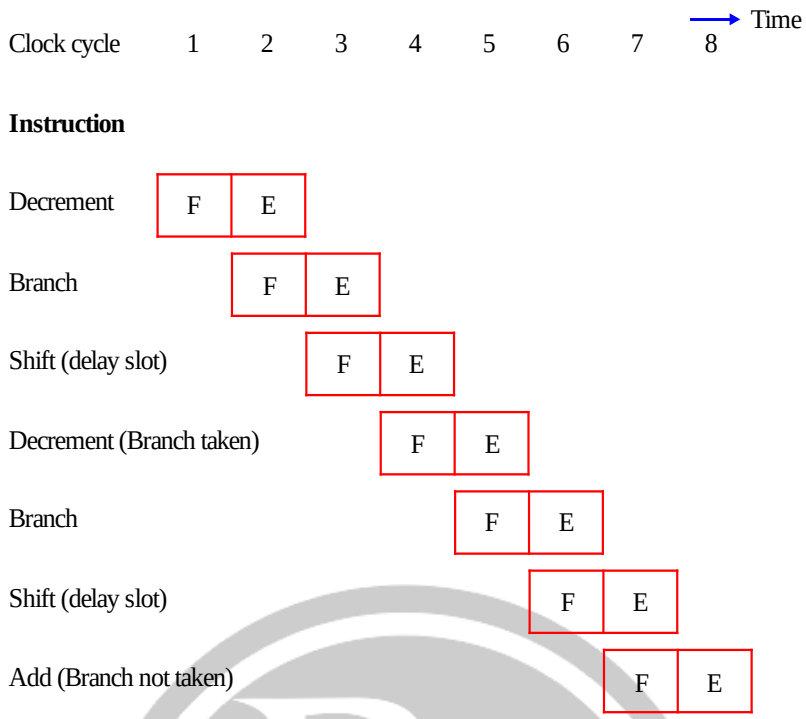
- In Figure the processor fetches instruction I_3 before it determines whether the current instruction, I_2, is a branch instruction. When execution of I_2 is completed and a branch is to be made, the processor must discard I_3 and fetch the instruction at the branch target. The location following a branch instruction is called a *branch delay slot.* There may be more than one branch delay slot, depending on the time it takes to execute a branch instruction. For example, there are two branch delay slots in Figure and one delay slot in Figure. The instructions in the delay slots are always fetched and atleast partially executed before the branch decision is made and the branch target address is computed.
- A technique called *delayed branching* can minimize the penalty incurred as a result of conditional branch instructions. The idea is simple. The instructions in the delay slots are always fetched. Therefore, we would like to arrange for them to be fully executed whether or not the branch is taken. The objective is to be able to place useful instructions in these slots. If no useful instructions can be placed in the delay slots, these slots must be filled with NOP instructions.

LOOP	Shift left	R1
	Decrement	R2
	Branch=0	LOOP
NEXT	Add	R1.R3

(a) Original program loop

LOOP	Decrement	R2
	Branch=0	LOOP
	Shift left	R1
NEXT	Add	R1.R3

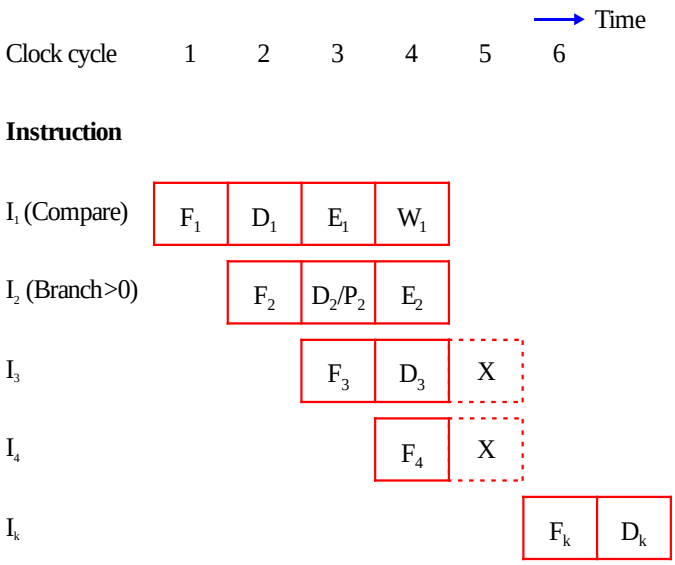
(b) Reordered instructions



- The effectiveness of the delayed branch approach depends on how often it is possible to reorder instructions as in Figure. Experimental data collected from many programs indicate that sophisticated compilation techniques can use one branch delay slot in as many as 85 percent of the cases. For a processor with two branch delay slots, the compiler attempts o find two instructions preceding the branch instruction that it can move into the delay slots without introducing a logical error. The changes of finding two such instructions are considerably less than the changes of finding one. Thus, if increasing the number of pipeline stages involves an increase in the number of branch delay slots, the potential gain in performance may not be fully realized.

7.5.2 Branch Prediction

- Another technique for reducing the branch penalty associated with conditional branches is to attempt to predict whether or not a particular branch will be taken. The simplest form of branch prediction is to assume that the branch will not take place and to continue to fetch instructions in sequential address order.
- Until the branch condition is evaluated, instruction execution along the predicted path must be done on a speculative basis. Speculative execution means that instructions are executed before the processor is certain that they are in the correct execution sequence.
- Hence, care must be taken that no processor registers or memory locations are updated until it is confirmed that these instructions should indeed by executed. If the branch decision indicates otherwise, the instructions and all their associated data in the execution units must be purged, and the correct instructions fetched and executed.



7.5.3 Dynamic Branch Prediction

- The objective of branch prediction algorithms is to reduce the probability of making a wrong decision, to avoid fetching instructions that eventually have to be discarded. In dynamic branch prediction schemes, the processor hardware assesses the likelihood of a given branch being taken by keeping track of branch decision every time that instruction is executed.
- In its simplest form, the execution history used in predicting the outcome of a given branch instruction is the result of the most recent execution of that instruction. The processor assumes that the nest time the instruction is executed, the result is likely to be the same.

7.6 INFLUENCE ON INSTRUCTION SETS

- We have seen that some instructions are much better suited to pipeline execution than others. For example, instruction side effects can lead to undesirable data dependencies. In this section, we examine the relationship between pipelined execution and machine instruction features. We discuss two key aspects of machine instructions – addressing modes and condition code flags.

7.6.1 Condition Codes

- In many processors, the condition code flags are stored in the processor status register. They are either set or cleared by many instructions, so that they can be tested by subsequent conditional branch instructions to change the flow of program execution. An optimizing compiler for a pipelined processor attempts to reorder instructions to avoid stalling the pipeline when branches or data dependencies between successive instructions occur. In doing so, the compiler must ensure that reordering does not cause a change in the outcome of a computation. The dependency introduced by the condition-code flags reduces the flexibility available for the compiler to reorder instructions.

- Consider the sequence of instructions in Figure, and assume that the execution of the Compare and Branch = 0 instructions proceeds as in Figure. The branch decision takes place in step E_2 rather than D_2 because it must await the result of the Compare instruction. The execution time of the Branch instruction can be reduced

```
Add          R1, R2
Compare      R3, R4
Branch = 0   . . .
```

(a) A program fragment

```
Compare      R3, R4
Add          R1, R2
Branch = 0   . . .
```

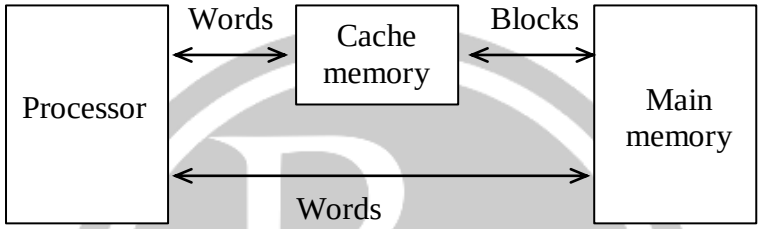
(b) Instructions reordered

❐❐❐

8 CACHE MEMORY

8.1. Introduction

- The speed of the main memory is very low in comparison with the speed of modern processors. For good performance an efficient solution is to use a fast cache memory.



  - It is the smallest and fastest memory component in the hierarchy.
  - By placing active portions of the program and data in a fast small memory, the average access time can be reduced.
  - The effectiveness of cache mechanism is based on principle called **"locality of reference"**
  - The Locality of reference states that, "The references to memory at any given interval of time tend to be confined within a few localized areas in memory. i.e. many instructions in localized areas of the program are executed repeatedly. It can be
    1. **Temporal:** It means that recently executed instruction is likely to be executed again very soon.
    2. **Spatial:** It means that instructions in close proximity to Cl recently executed instruction,

       Example: loops, nested loops, procedure calls. Etc.

- The performance of cache is measured using Hit ratio

$$\text{Hit ratio} = \frac{\text{no. of hits}}{\text{Total CPU references}} \times 100$$

$$\text{Hit ratio} = \frac{\text{no. of hits}}{\text{no.of hits} + \text{no.of misses}} \times 100$$

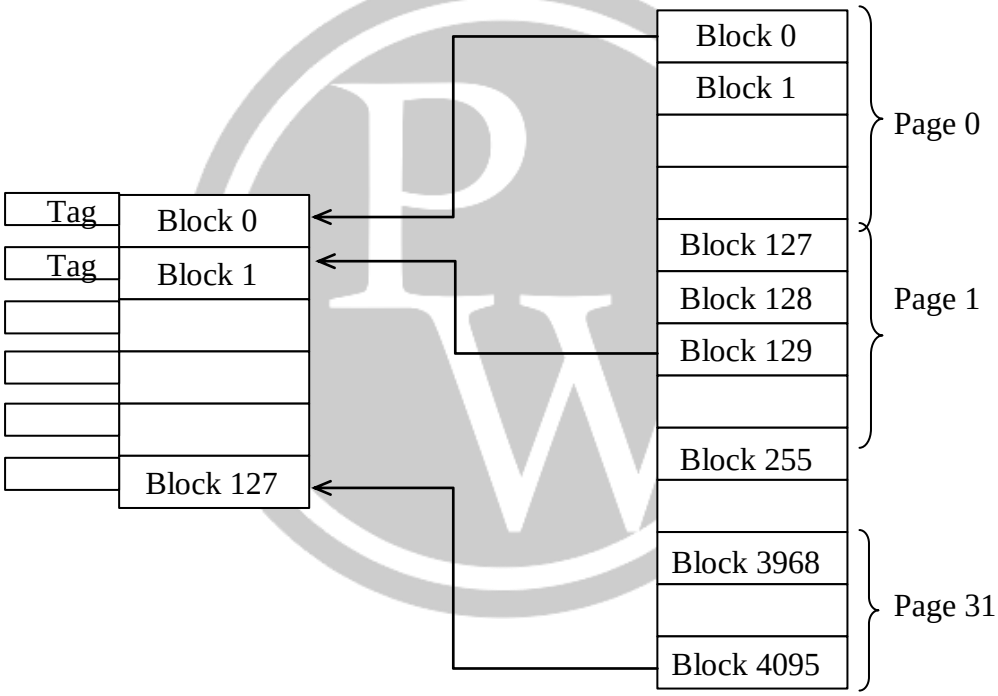
- The performance of cache can be analysed with the following characteristics.
  (1) Cache size (Small in KB's)
  (2) Block or line size
  (3) No. of levels of cache
  (4) Cache mapping process.
  (5) Cache replacement algorithm
  (6) Cache updating scheme.

8.2. Cache Mapping Functions

- Consider a cache consisting of 128 blocks of 16 words cache, assume the main memory is addressable by a 16-bit address. The main memory is addressable by a 16 –bit address. The main memory has 64K words viewed as 4K blocks of 16 words each.
- The various mapping functions are
  (i) Direct mapping
  (ii) Associative mapping
  (iii) Set – Associative mapping

8.2.1 Direct Mapping

- The simplest way to determine cache locations in which to store memory blocks.
- Each block from the main memory has only one possible location in cache.
- Block j of the main memory maps to block j mod n. Where n is the no. of blocks in the cache.



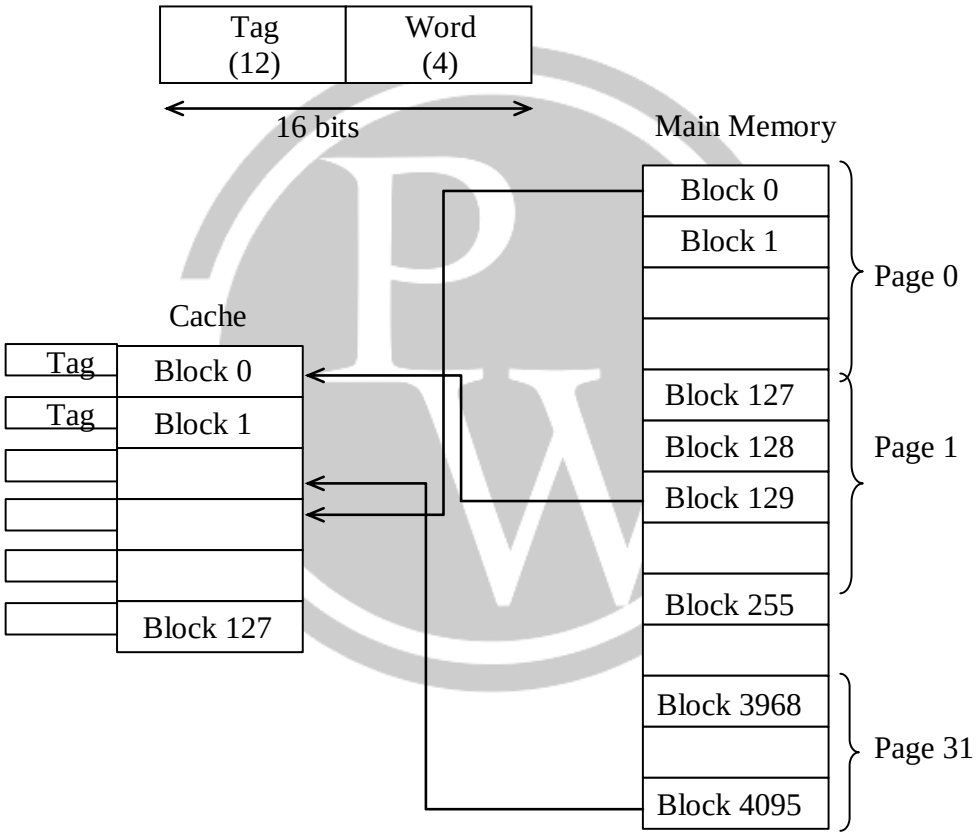
- If the processor needs to access same memory locations from two different pages of the main memory frequently, hen miss & will be more.
- Easy to implement but hit ratio is less.
- To implement this the address is divided into Three fields.
- Example :

Tag	Block	Word
5	7	4

- As execution proceeds, the higher order tag bits of the address are compared with the tag bits associated with that cache location. If they match, then the desired word is in that block of cache. If there is no match, then miss will occur.
- Requires only one comparison.

8.2.2 Associative Mapping

- In which a main memory block can be placed into any cache block position.
- It gives complete freedom in choosing the cache location in which to place the memory block. Thus the space in the cache can be used effectively.
- A new block that has to be brought into the cache has to replace an existing block if the cache is full.
- The cost is high, because of need to search all tag patterns to determine whether a given block is in the cache, i.e. comparison circuit is more complex.
- Requires maximum of n comparisons. Where n is the number of cache blocks.
- The no. of tag comparators = no. of cache blocks.
- The address is divided into 2 fields.

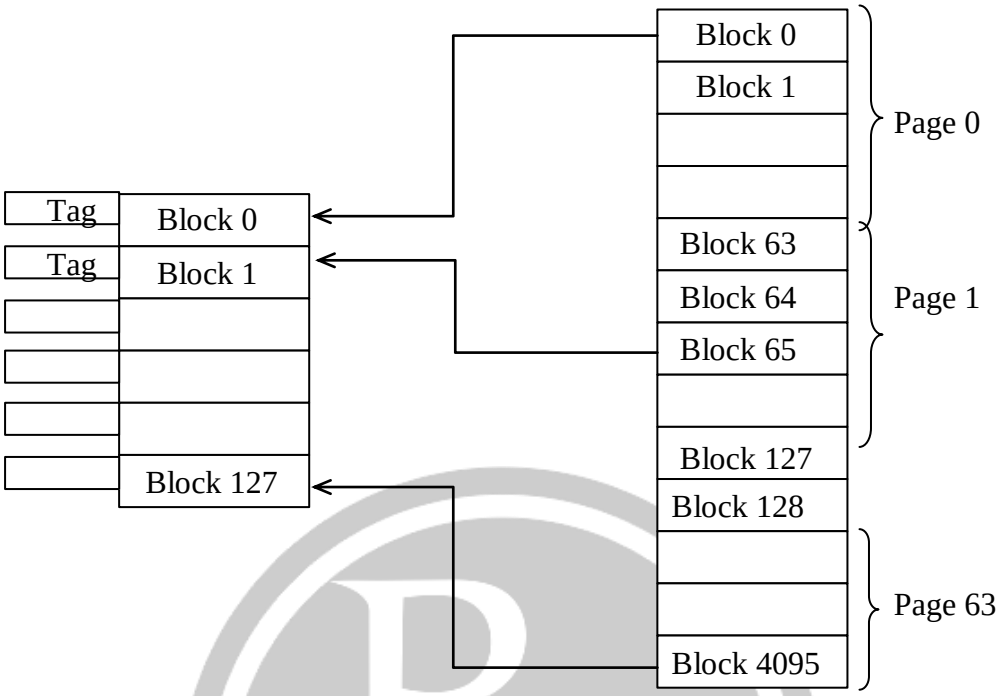


- We need an algorithm to select the block to be replaced.

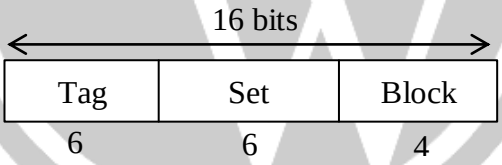
8.2.3 Set – Associative Mapping

- For efficiency, a combination of direct and associative mapping techniques can be used.
- Blocks of the cache are grouped into sets, and the mapping allows a block of the main memory to reside in any block of a specific set. i.e. contains several groups of directed mapped caches in parallel.
- The contention problem of the direct mapped method is eased by having a few choices for block placement.

- The hardware cost is reduced for comparing tags unlike associative mapping.
- A block can occupy either of blocks in the set.



- A cache that has K blocks per set, then it is referred as K-way set-associative cache.
- The address is divided into 3- fields.



- Number of tag comparisons is equal to number of blocks in the set.

8.3. Cache Replacement Policy

- In direct mapped cache, the position of each block is predetermined hence no replacement strategy exists.
- In Associative and set – Associative caches there exists some flexibility, when a new block is to be brought into the cache and all positions that it may occupy are full. Hence replacement strategy exists.
- The replacement policies are aimed for reducing the penalty (miss) to feature references.

The various block replacement policies are

(1) FIFO: The block which enters first will be the candidate for replacement. i.e. Assumes that, since it has spent long time in cache all the references to it are slightly exhausted. Implemented using queue data structure.

(2) LRV: The block in the se t which has been in the cache longest with no references to it is selected for the replacement. (Least recently used).

(3) LFU: The block in the set which has the fewest references is selected for replacement. (Least frequently used).3

(4) Random: No specific criteria for replacement of any block. The existing blocks are replaced randomly.

8.4. Cache Updation Policy

- The two cache updation schemes employed are

➢ **Write – through:** The simplest and commonly used approach. Updation of cache and main memory are done simultaneously.

  ○ Main memory contains the same data as the cache. Which is important for DMA transfers.

➢ **Write–back:** In this method, only the cache location is updated during a write operation. The location is then marked by a flag so that later when the word is removed from the cache it is copied into main memory.

  ○ i.e. updation of main memory is postponed until the cache block selected for replacement.

8.5. Cache Memory & Arrays

An array is a homogeneous collection of data items stored in contiguous memory locations either in row major order or column major order.

Example:

$$\begin{bmatrix} 3 & 1 & 2 \\ 5 & 6 & 9 \\ 8 & 4 & 7 \end{bmatrix}$$

Row major:

00	01	02	10	11	12	20	21	22
3	1	2	5	6	9	8	4	7

Col major:

00	10	20	01	11	21	02	12	22
3	5	8	1	6	4	2	9	7

Q.1. Consider an array is A[100] and each element occupies 4 – words, 32 – word cache is used and it is divided into 8 – word blocks (a) what is the hit ratio for the following instruction.

(a) For (i = 0 ; i < 100 ; i ++)
A [i] = A [i] + 10 ;

Ans.

	R	W
A[0]	M	H
A[1]	H	H
A[2]	M	H
A[3]	H	H

A(0)
A(1)
A(2)
A(3)

(b) How many times block '0' is modified in the cache memory.

Ans. 0, 8, 16, - - - - - 80, 88, 96. ⇒ 13 times.

(c) How many times block '0' is replaced.

Ans. 12 times.

❑❑❑

9 MAIN MEMORY

9.1. Introduction

- The main memory is the central storage unit in a computer system. The principal technology used for the main memory is based on semiconductor integrated circuits. Main memory is made up of RAM and ROM chips. Different types of integrated circuit memories are given in Table 1.
- Integrated circuit RAM chips are available in two possible operating modes, static and dynamic. The static RAM consists of interval flip flops that store the binary information. The stored information remains valid as long as power is applied to the unit i.e. ROM is volatile. The dynamic RAM stores the binary information in the form of electric charges that are applied to capacitors.

Memory	Access	Volatile	Features
Read/Write	Random or serial	Yes	Data can be read or written with equal case. Used whenever data is needed fast and often, and is frequently changed.
Read-only (ROM)	Random	No	(a) Data is stored permanently by a masking step during manufacture. (b) Used whenever data is not to be changed as in fixed tables, constants or computer instruction sets.
Programmable Read-only (PROM)	Random	No	(a) Can be programmed after IC is manufactured. (b) Data is written by opening fusible metal links or P-N junction's wit high currents. (c) Can only be written into once. Same use as a ROM, but is cheaper in small quantities.
Re-programmable read-only	Random	No	Are ROMs that can be written into many times. Are really "read-mostly" rather than "read-only" memories Two main types: (a) Those in which writing is by voltage application and erasing (all data at once) by exposing chip to ultra-violet radiation through a window on the IC (b) Those in which reading and writing is electrical. Data remains stored even with no power applied through the use of MNOS transistors. Used where frequent or at least more than one change in a program is required as in debugging a program.
Content-addressable (CAM)	Random	Yes	This extracts all data in an address when a part of the contents of that address match a specified number. Used in associative memories to obtain stored data related in some way to the input data.

Charge-coupled device (CCD)	Serial	Yes	(a) Digital input is converted to charge and stepped through a shift register. (b) Requires refresh circuitry to present data loss.
Programmable logic array (PLA)	Random	No	(a) Is a memory structure that is mask or field (FPLA) programmed as is a ROM. However, it implements logic function. (b) Inputs are functions of the input variables and the logic operation stored in the address. (c) Is more flexible than random (hard-wired separate IC) circuits because PLAs and FPLAs are programmable.

Refreshing is done by cycling through the words every few milliseconds to restore the decaying charge. The dynamic RAM offers reduced power consumption and larger storage capacity in a single memory chip. The static RAM is easier to use and has shorter read and write cycles. The Table 2 shows the summary of various types of integrated circuit memories.

(a) Types of Memory Access	
Random access	Any address can be accessed at random, that is, without going through other address first. Data retrieval time is relatively fixed.
Serial access	Typified by shift register or charge-coupled-device (CCD) memories. Data retrieval is serial in a fixed order. All data ahead of required data must be read first.
(b) Static verses Dynamic Storage	
Static storage	Data is stored in flip-flops or other memory cells in which the data does not deteriorate with time.
Dynamic storage	Data is stored in leaky capacitors so that refresh circuitry is required to prevent data loss.

9.2. Memory Interface

A computer uses memory capacity of 512 bytes of RAM and 512 bytes of ROM. The IC sizes of RAM and ROM are 128 x 8 and 512 × 8 bits respectively.

(a) Find the number of RAM ICs.

(b) Find the number of ROM ICs.

(c) Give the memory map of the system

(d) Mention the size of decoder.

(e) Give the diagram of memory connection to the CPU.

Solution:

(a) The number of RAM ICs $= \frac{\text{Total'RAM size}}{\text{RAM IC size}}$

$= \frac{512\times 8}{128\times 8}$

$= 4$

(b) The number of ROM ICs $= \frac{\text{Total 'ROM size}}{\text{ROM IC size}}$

$= \frac{512 \times 8}{512 \times 8}$

$= 1$

(c) The memory map of this system is given by Table. 3

Component	Hexadecimal address	Address bus											
		10	9		8	7	6	5		4	3	2	1
RAM 1	0000–007F	0	0		0	×	×	×		×	×	×	×
RAM 2	0030–00FF	0	0		1	×	×	×		×	×	×	×
RAM 3	0100–017F	0	1		0	×	×	×		×	×	×	×
RAM 4	0180–01FF	0	1		1	×	×	×		×	×	×	×
ROM	0200–03FF	1	×		×	×	×	×		×	×	×	×

(d) 2 × 4 decoder

Each RAM receives the seven low-order bits of the address bus to select one of 128 possible bytes. The particular RAM chip selected is determined from lines 8 and 9 in the address bus. This is done through a 2 × 4 decoder. The outputs of decoder are given to the CSI inputs in each RAM chip. Thus when address lines 8 and 9 are equal to 00, the first RAM chip is selected when 01, the second RAM chip is selected and so on.

(e) The connection of memory chips to the CPU is shown in Fig. RAM and ROM chips are connected to a CPU through the data and address buses. The low order lines in the address bus selected the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs.

❐❐❐

10 SECONDARY STORAGE

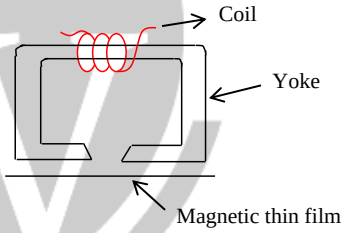
10.1 Introduction

- The cost per bit of stored information is high in c semi-conductor memories (main memory). This limits the use of these memories for large storage. Large storage requirements of most of computers are economically fulfilled by magnetic disks, magnetic tapes and optical disk memories. These memories are referred as secondary storage devices.

10.2 Magnetic Disks

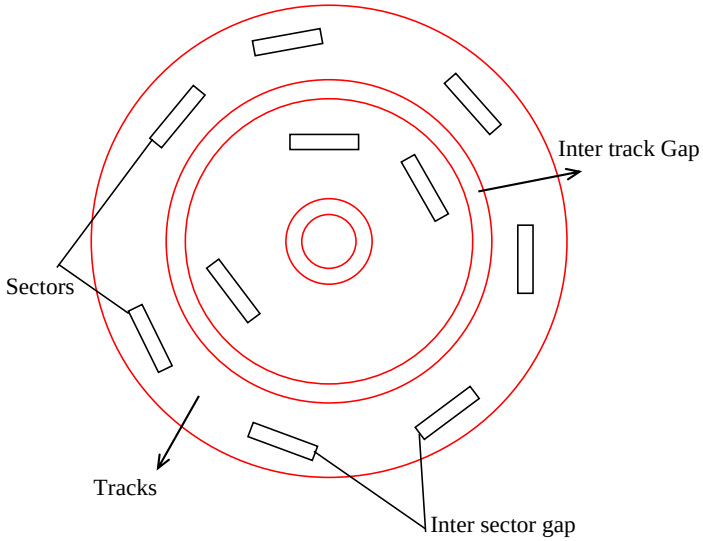
- A magnetic disk is a thin, circular metal plate. It is coated with a thin magnetic film usually on both sides.
- Digital information is stored on the magnetic disk by magnetizing the magnetic surface in a particular direction.
- Magnetic disks are semi random access memories.

The head consists of a magnetic yoke and the magnetizing coil. The Digital information can be stored on the film by applying current pulses of suitable polarity.



10.2.1. Data organization & Formatting

- The data on the disk is organized in a concentration set of rings. These concentric set of rings are called tracks.
  - ✓ Each track has same width as head and adjacent tracks are separated by gaps.
  - ✓ Each track is divided into manageable units called sectors.
  - ✓ Each sector stores a block of data which can be transferred to or from the disk.
  - ✓ The universal size of sector is 512 bytes.
  - ✓ The R/W head is capable of reading a sector from disk and writing a sector to disk.
  - ✓ Placing R/W head on desired track is random access and reading concerned sector is serial. Hence disks are semi random access devices.



10.2.2. Physical Characteristics

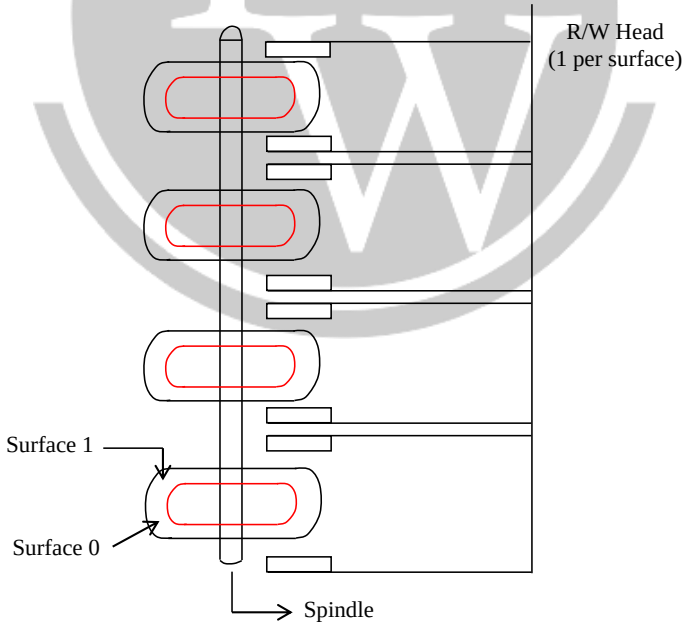
(i) Head Motion - Fixed head (one per track)
Movable head (one per surface)

(ii) Disk portability - Non removable or Removable

(iii) Sides - Single sided, Double – sided.

(iv) Disk/surface - Single surface, Multiple – surfaces

(v) Head Mechanism - Contact, fixed Gap, Aerodynamic Gap.



A vertical set of all of the tracks with the same number on each surface of a diskette or hard disk is called "cylinder."

(vi) Platters - Single platter,
Multiple platters – some disk
Accommodate multiple platters
Stacked vertically a fraction of an inch apart.

(1) If all the tracks are having constant capacity then the disk moves with "constant angular capacity" and "recording density" varying from track to track.

(2) If each track is having variable capacity then the disk is moving with "constant linear velocity"

- Placing the control information in the sector is called as formatting.
- The disk controller is the hardware interface to control the operation of a disk drive. Used to control more than one drive. The major functions are seek, read, write and error checking.
- The start and end of each sector is determined by the control data stored in the sector.
- Disk performance parameters:

Seek Time: It is the time required to more the disk arm to the required track.

Rotational Delay: The time taken for the beginning of sector to reach. (latency)

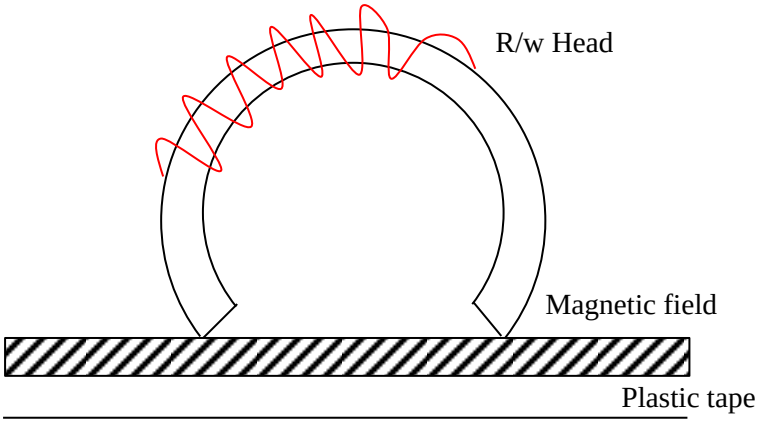
Access time: The sum of seek time and the rotational delay.

- The average time to disk access is
  $t_{avg} = t_{seektime} + t_{rotational\ delay} + t_{data\ transfer} + t_{over\ head}$
- Maximum Recording Density (P) = No. of Bytes/cm
- Data transfer rate (D) = Number of Bytes/Sec.

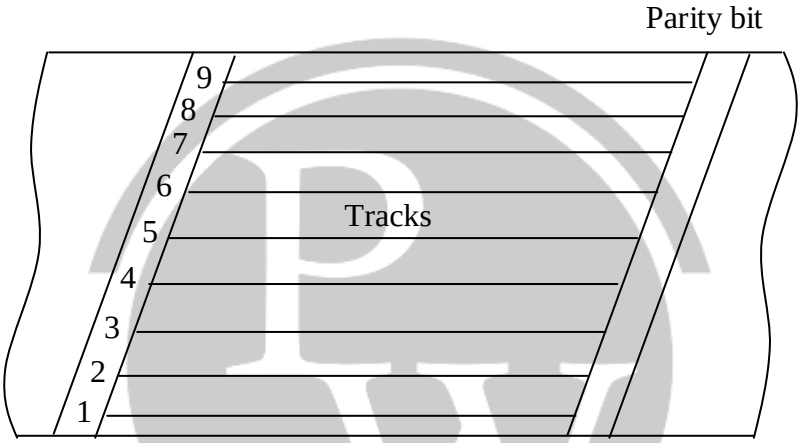
10.3 Secondary Storage Devices

10.3.1. Magnetic Tapes

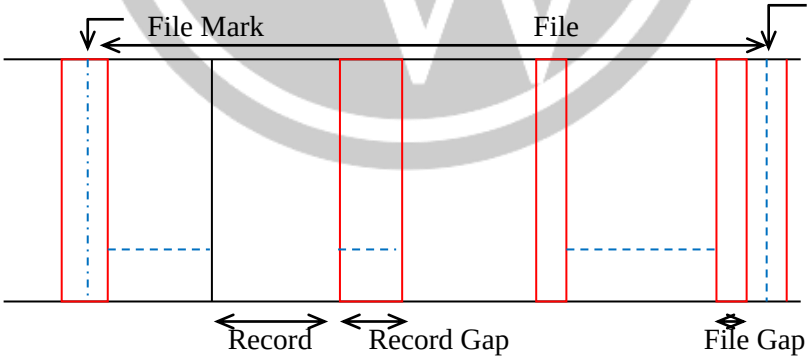
- Magnetic tape is one of the most popular storage medium for large data that are sequentially accessed and processed.
- The tape is formed by depositing magnetic film on a very thin and ½ or ¼ inch wide plastic tape.
- Usually, iron oxide is used as a magnetizing material.
- The information is recorded on the tape with the help of read/write head. It magnetizes or non magnetizes tiny invisible spots (1's & 0's).
- Usually 7 or a bits are recorded (a character) in parallel across the width of the tape, perpendicular to the direction of motion. A separate R/Wo head is provided for each bit position on the tape.



- Data on the tape are organized in the form of records separated by gaps.
- A set of related records constitutes a file.



- The data on a 9th track (a) tape is stored in parallel consists of data byte and a parity bit.



- Data transfer takes place when the tape is moving at constant velocity relative to a read/write head.
- Hence the maximum data transfer rate depends largely on the storage density along the tape and speed of the tape.
- The file mark is used as header or identifier.
- The information on the magnetic tape is organized into blocks. These blocks have a fixed length and separated by gap between them.
- If the block length is B_L and inter block gap length is G_L, then the utilization factor of the tape (u) is given by

$$u = \frac{B_L}{B_L + G_L}$$

- The unit of data transfer is a record.
- The tape is moving with a linear velocity and Read/write head is constant.
- Let L is the length of the tape,
  - N is the number of parallel tracks,
  - P is the constant recording density.

(i) Capacity of the tape

$$C = L \times N \times P$$

(ii) Let V is the linear velocity of the tape, then the data transfer rate

$$D = V * N * P$$

✓ With utilization factor, the effective data transfer rate is

$$D_{eff} = D * \frac{B_L}{B_L + G_L}$$

✓ Due to inter block gap and time needed to start and stop and tape between accesses, the effective data transfer rate d_{eff} is actually less than the maximum rate d.

$$d_{eff} = \frac{t_D \times d}{t_D + t_G + t_{ss}}$$

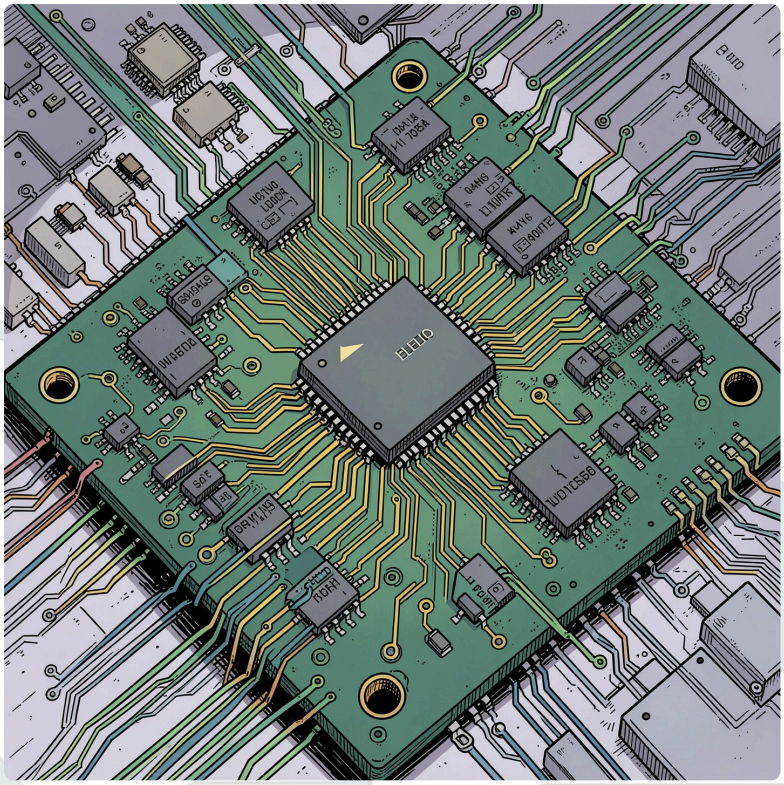
t_D = time to scan a data block

t_G = time to scan the inter block gap.

T_{ss} = time to start and stop the tape.

.

❐❐❐



Computer Organization & Architecture — GATE 2027 Master Sheet

GATE WALLAH | IIT MADRAS STANDARD | 2007–2026 PYQS

Welcome to the most comprehensive, high-yield study guide for **Computer Organization and Architecture (COA)** — designed specifically for GATE 2027 aspirants and DBATU B.Tech final-year students aiming for IIT Madras standard. This document covers every topic from the GATE syllabus with real-world analogies, mathematical depth, trap warnings, worked examples, and 50+ GATE-style practice questions with complete solutions.

COA has consistently contributed **8–12 marks** in GATE CS/IT papers from 2007 to 2024, making it one of the highest-scoring subjects when mastered correctly. The topics span from machine instructions and addressing modes, through ALU design and control units, to memory hierarchy, I/O interfaces, and instruction pipelining — each carrying significant weightage in recent years.

What makes this guide different? Every concept is explained using **memorable real-world analogies** (restaurant orders, train ticketing, banking transactions), followed by precise mathematical rules, GATE trap warnings drawn from actual student mistakes, and step-by-step worked examples. The practice questions are modelled on actual GATE patterns — including NAT, MCQ, MSQ, and tricky multi-concept problems — with explanations that tell you not just the answer but WHY every wrong option is wrong.

This document covers the complete GATE COA syllabus: **Machine Instructions & Addressing Modes**, **ALU Design & Data Path**, **Control Unit (Hardwired & Microprogrammed)**, **Memory Hierarchy & Cache Mapping**, **I/O Interface (Interrupt & DMA)**, and **Instruction Pipelining & Hazards**. Each topic includes previous year GATE questions from 2007 onwards, DBATU syllabus alignment, and standard textbook references (Hamacher, Stallings, Patterson & Hennessy).

Whether you are starting from scratch or revising in the last 90 days, this sheet is your single source of truth. Read it once for understanding, twice for retention, and thrice for mastery. Let's crack GATE 2027 together! 🚀

Made with GAMMA

Machine Instructions & Addressing Modes

TOPIC 1 | WEIGHTAGE: 3–5 MARKS | GATE 2008–2024

🍕 High-Yield Concept & Intuition — The Restaurant Order Analogy

Think of a **machine instruction** as a restaurant order slip. The **opcode** is WHAT to cook (BIRYANI = ADD, ROTI = SUB), and the **operands** are WHERE the ingredients are stored. Now, **addressing modes** answer: "How do I find those ingredients?"



Immediate Mode

Ingredient is written ON the order slip itself. No searching needed. Example: `ADD R1, #5` — value 5 is in instruction.



Direct Mode

Order slip has exact shelf number. Go directly there. Example: `ADD R1, 1000` — address 1000 holds operand.



Register Indirect

Order says "check the notebook in drawer R1." Register holds the ADDRESS, not the value.



Stack Mode

Everything is on a stack — push, pop, operate on top. No address needed. Used in stack machines.

🔬 Deep-Dive Core Mechanics & Mathematical Rules

An instruction format has three fields: **Opcode** (operation type), **Address Field** (operand location), and **Mode Field** (addressing mode). The **Effective Address (EA)** is the actual memory location accessed.

Addressing Mode	Effective Address Formula	GATE Example
Immediate	EA = Value in instruction	`ADD R1, #10`
Direct	EA = Address field (A)	`LDA 2000`
Register	EA = Register R	`ADD R1, R2`
Register Indirect	EA = (R)	`LDA (R1)`
Base Register	EA = Base + Displacement	Relocation support
Indexed	EA = Index Register + A	Array access
Relative (PC)	EA = PC + Displacement	Branch instructions
Auto Increment	EA = (R)+ ; R ← R + d	Stack pop
Auto Decrement	EA = -(R) ; R ← R - d	Stack push

Instruction Formats: Zero-address (stack), One-address (accumulator), Two-address, Three-address. More addresses = faster execution but larger instruction size. Three-address needs $3 \times \lceil \log_2 n \rceil + \text{opcode bits}$.

⚠️ GATE Trap Warnings

Trap 1: Indirect vs Register Indirect

Indirect: Memory location holds address → **two memory accesses. Register Indirect:** Register holds address → **one memory access.** GATE 2014 asked this distinction explicitly.

Trap 2: PC-Relative Addressing Range

For n-bit displacement in 2's complement: range is -2^{n-1} to $2^{n-1} - 1$ bytes from current PC. GATE 2017: 16-bit instruction, 6-bit opcode → 10-bit address → ±512 bytes range.

Trap 3: Auto-Increment/Decrement Value

Increment/decrement amount = **size of data type** (1 byte for char, 2 for short, 4 for int). GATE 2011: `LW R1, (R2)+` with 32-bit word → R2 increments by 4.

📝 Worked Examples

Example 1 — Effective Address Calculation: Instruction `ADD R1, X(R2)` where R2 = 2000, X = 500 (displacement), memory[2500] = 100. Indexed mode: EA = R2 + X = 2000 + 500 = 2500. Result = R1 + memory[2500] = R1 + 100.

Example 2 — PC-Relative Branch: 16-bit instruction, PC = 3000, branch displacement = -50 (signed). Target = PC + displacement = 3000 + (-50) = 2950. PC is already incremented to next instruction before adding displacement.

🎯 5 GATE-Style Practice Questions

1

Q1 — Conceptual (1 Mark)

In which addressing mode is the operand specified *within* the instruction itself?
A) Direct B) Register C) Immediate D) Register Indirect

Answer: C — Immediate mode embeds the value directly in the instruction field.

2

Q2 — NAT (1 Mark)

A processor has 64 registers and 128 instructions. Each instruction has opcode, two register operands, and 6-bit immediate. Minimum instruction size = ? bits

Answer: 26 bits — Opcode: 7 bits ($128 = 2^7$), each register: 6 bits ($64 = 2^6$), immediate: 6 bits. Total = 7 + 6 + 6 + 6 = **25 bits → rounded to 26 for alignment.**

3

Q3 — MCQ (2 Marks)

Which addressing modes support program relocation?
A) Immediate B) Base Register C) PC-Relative D) Direct

Answer: B, C — Base register and PC-relative use offsets from a movable base, enabling relocation.

4

Q4 — MSQ (2 Marks)

Which modes require memory access to fetch the operand?
A) Register B) Direct C) Register Indirect D) Immediate

Answer: B, C — Direct accesses memory at address A; Register Indirect accesses memory at address in register.

5

Q5 — Tricky GATE (2 Marks) [GATE 2017]

A RISC processor: 5 stages, 100 instructions, each needs 1 opcode + 2 source regs + 1 dest reg + 1 immediate. Regs = 64, immediate = 12-bit signed. Minimum bits per instruction?

Answer: 37 bits — Opcode: 7 bits ($100 \rightarrow 2^7$), each reg: 6 bits ($64 \rightarrow 2^6$) × 3 = 18, immediate: 12 bits. Total = 7 + 18 + 12 = **37 bits.**

Made with GAMMA

ALU Design & Data Path

TOPIC 2 | WEIGHTAGE: 2–4 MARKS | GATE 2009–2023

🔧 High-Yield Concept & Intuition — The Calculator Factory

Imagine the ALU as a **calculator factory**. Raw materials (operands) enter through conveyor belts (buses), workers (functional units) perform operations, and finished products (results) exit. The **Data Path** is the entire factory floor layout — which conveyor connects to which machine.

ALU Core Operations

- Arithmetic: ADD, SUB, MUL, DIV
- Logical: AND, OR, NOT, XOR
- Shift: Logical, Arithmetic, Rotate
- Compare: Set flags (Z, N, V, C)

Data Path Components

- **Registers:** Temporary storage (R0–Rn)
- **ALU:** Performs operations
- **MUX:** Selects input source
- **Bus:** Communication highway
- **Control Signals:** Tell each component what to do

🔬 Deep-Dive Core Mechanics & Mathematical Rules

Ripple Carry Adder: N-bit RCA delay = $N \times t_{carry}$. For 32-bit: 32 gate delays. **Carry Lookahead Adder (CLA):** Generate $G_i = A_i \cdot B_i$, Propagate $P_i = A_i \oplus B_i$. Carry: $C_{i+1} = G_i + P_i \cdot C_i$. CLA delay = $O(\log N)$.

Booth's Algorithm (GATE favourite!): Multiply signed 2's complement numbers. Rules: (0,0) → shift; (0,1) → add M; (1,0) → subtract M; (1,1) → shift. Reduces partial products for consecutive 1s.

Restoring Division

Shift left, subtract divisor. If result negative → restore (add back), quotient bit = 0. Else quotient bit = 1. Slow but simple.

Non-Restoring Division

If negative → add divisor next step instead of restoring. Faster — no restore cycle. GATE 2016 asked comparison.

IEEE 754 Format

Single: 1 sign + 8 exp (bias 127) + 23 mantissa. Double: 1 + 11 (bias 1023) + 52. Value = $(-1)^s \times 1.M \times 2^{E-127}$.

⚠️ GATE Trap Warnings

Trap 1: Overflow vs Carry

Carry flag = unsigned overflow. **Overflow flag** = signed overflow (carry into sign bit ≠ carry out of sign bit). GATE 2012: ADD two negative numbers giving positive → overflow, not carry.

Trap 2: Booth's Recoding Count

For n-bit multiplier, Booth's performs exactly **n/2 additions maximum** (not n). Consecutive 1s are grouped. GATE 2018: 16-bit → max 8 additions.

Trap 3: IEEE 754 Special Values

Exp = 0, Mantissa = 0 → Zero. Exp = 255, Mantissa = 0 → Infinity. Exp = 255, Mantissa ≠ 0 → NaN. Exp = 0, Mantissa ≠ 0 → Denormalized. GATE 2020 tested this.

📝 Worked Examples

Example 1 — CLA Carry Calculation: 4-bit CLA, A = 1011, B = 0110, $C_0 = 0$. G_0=0, G_1=0, G_2=1, G_3=0. P_0=1, P_1=0, P_2=0, P_3=1. $C_1 = 0+1\cdot0 = 0$. $C_2 = 0+0\cdot0 = 0$. $C_3 = 1+0\cdot0 = 1$. $C_4 = 0+1\cdot1 = 1$. Sum = 10001 (17 in decimal, correct: 11+6=17).

Example 2 — Booth's Multiplication: M = 5 (0101), Q = -3 (1101 in 4-bit 2's comp). $Q_{-1} = 0$. Step 1: $Q_0Q_{-1} = 10$ → A = A - M. Shift right arithmetic. Repeat 4 times. Final AQ = product = -15.

🎯 5 GATE-Style Practice Questions

1

Q1 — Conceptual (1 Mark)

In a 4-bit CLA, the carry-out C_4 depends on:
A) Only G_3, P_3 B) All G_0–G_3, P_0–P_3 C) Only P_0–P_3 D) Only G_0, G_1

Answer: B — $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

2

Q2 — NAT (1 Mark)

A 32-bit ripple carry adder has carry propagation delay = 2 ns per stage. Total worst-case delay = ? ns

Answer: 64 ns — 32 stages × 2 ns = **64 ns.**

3

Q3 — MCQ (2 Marks) [GATE 2012]

Overflow occurs in 2's complement addition when:
A) Carry out of MSB = 1 B) Carry into MSB ≠ Carry out of MSB C) Result is negative D) Both operands same sign

Answer: B — Signed overflow: carry-in to sign bit ≠ carry-out from sign bit.

4

Q4 — MSQ (2 Marks)

Which are advantages of CLA over RCA?
A) Reduced delay B) Simpler circuit C) O(log n) delay D) Less hardware

Answer: A, C — CLA has O(log n) delay vs O(n) for RCA, but uses more hardware.

5

Q5 — Tricky GATE (2 Marks) [GATE 2020]

IEEE 754 single precision: hex representation 0x40400000. Decimal value = ?

Answer: 3.0 — Sign=0, Exp=10000000_2=128, E=128-127=1, Mantissa=100000... → $1.100\times2^1 = 1.5\times2 = \mathbf{3.0}$

Made with GAMMA

Control Unit — Hardwired vs Microprogrammed

TOPIC 3 | WEIGHTAGE: 2–3 MARKS | GATE 2010–2022

🎛️ High-Yield Concept & Intuition — The Orchestra Conductor

The **Control Unit** is the orchestra conductor. Musicians (ALU, registers, memory) know how to play, but need signals telling them *when* and *what* to play. A **Hardwired CU** is like a conductor with a fixed memorised score — lightning fast but can't learn new music. A **Microprogrammed CU** carries a sheet music book — slower but can handle any symphony by loading new pages.

Hardwired Control
Logic gates + flip-flops. Control signals = Boolean functions of opcode + timing signals. Fastest but inflexible. Used in RISC.

Microprogrammed Control
Control memory stores microinstructions. Each machine instruction = sequence of micro-ops. Flexible, easier to modify. Used in CISC.

🔬 Deep-Dive Core Mechanics & Mathematical Rules

Hardwired CU: Uses **Sequence Counter + Decoder + PLA/PAL**. Control signals are minimised Boolean expressions. Speed = 1–2 gate delays. **Microprogrammed CU:** Control Memory (ROM) stores microinstructions. Each microinstruction has **Control Field** (control signals) + **Next Address Field** (sequencing).

Feature	Hardwired	Microprogrammed
Speed	Very fast	Slower (control memory access)
Flexibility	Fixed, hard to modify	Easy to modify/update
Cost	High design cost	Lower design cost
Complexity	Complex for CISC	Handles complex ISAs easily
Used in	RISC processors	CISC processors
Debugging	Difficult	Easier

Vertical vs Horizontal Microinstructions: Vertical = encoded (fewer bits, slower decode). Horizontal = one bit per control signal (more bits, faster). **Control Memory Size** = $2^{\text{address bits}} \times \text{microinstruction width}$.

⚠️ GATE Trap Warnings

Trap 1: Nano Memory Confusion
Two-level microprogramming: **Control Memory → Nano Memory**. Nano stores frequently used micro-op patterns. GATE 2019: Control memory = 1024 × 20 bits, Nano = 256 × 8 bits → total = 1024×20 + 256×8.

Trap 2: Horizontal vs Vertical Bit Count
Horizontal: **1 bit per control signal** (100 signals = 100 bits). Vertical: **encoded** (100 signals → ~7 bits per field). GATE 2015: 512 control signals → horizontal = 512 bits, vertical ≈ 9 bits per field.

Trap 3: Microinstruction Cycle Time
Microprogrammed CU cycle time = **control memory access time** (not gate delay). Hardwired = gate delay. This is why hardwired is faster. GATE 2011 tested this comparison.

📝 Worked Examples

Example 1 — Control Memory Size: 256 control signals, horizontal microinstruction, 4 status bits for branching, 1024 microinstructions. Width = 256 (control) + 10 (address, since 2^{10}=1024) + 4 (status) = 270 bits. Size = 1024 × 270 bits = 270 Kbits.

Example 2 — Nano Memory Savings: Without nano: 1000 × 40 bits = 40,000 bits. With nano: Control memory = 1000 × 10 bits (nano address) + Nano = 100 × 40 bits. Total = 10,000 + 4,000 = 14,000 bits. Saving = 26,000 bits.

🎯 5 GATE-Style Practice Questions

1

Q1 — Conceptual (1 Mark)
Which control unit type is preferred for RISC processors?
A) Microprogrammed B) Hardwired C) Nano D) Vertical
Answer: B — RISC has simple, fixed instructions → hardwired gives maximum speed.

2

Q2 — NAT (1 Mark) [GATE 2015]
A processor has 512 control signals. Horizontal microinstruction. Minimum bits for control field = ?
Answer: 512 — Horizontal = 1 bit per signal → **512 bits.**

3

Q3 — MCQ (2 Marks)
Vertical microprogramming is characterised by:
A) One bit per control signal B) Encoded control signals C) Faster execution D) Larger control memory
Answer: B — Vertical uses encoded fields, reducing width but requiring decode logic.

4

Q4 — MSQ (2 Marks) [GATE 2019]
Advantages of microprogrammed control over hardwired:
A) Easier to add new instructions B) Faster execution C) Easier debugging D) Less control memory
Answer: A, C — Microprogrammed is flexible and easier to debug, but slower and uses more memory.

5

Q5 — Tricky GATE (2 Marks)
Control memory: 4096 words × 24 bits. Each microinstruction: F1(4 bits), F2(4 bits), F3(4 bits) for control signals; CD(2 bits) for status; BR(2 bits) for branch; AD(8 bits) for address. Number of control signals = ?
Answer: 32 — F1, F2, F3 each 4 bits → each field can encode 2^4 = 16 signals. If each field controls separate group: 16 + 16 + 16 = **48 signals maximum** (vertical encoding). If asked for total unique: depends on grouping.

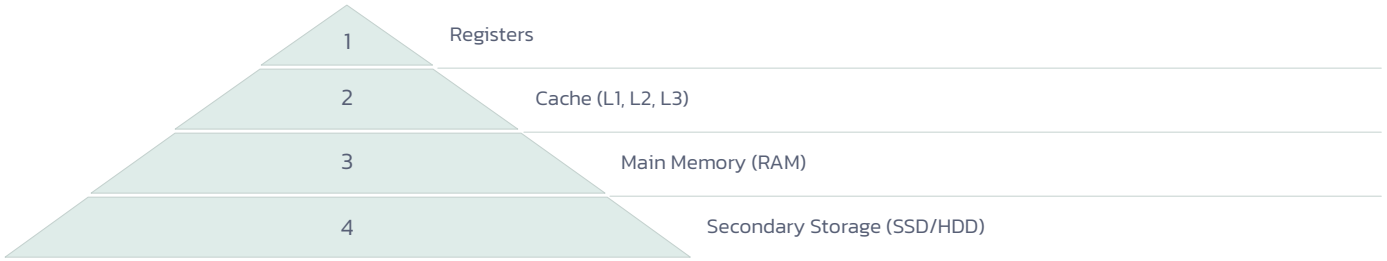
Made with GAMMA

Memory Hierarchy — Cache, Main Memory & Secondary Storage

TOPIC 4 | WEIGHTAGE: 4–6 MARKS | GATE 2007–2024

🏢 High-Yield Concept & Intuition — The Office Filing System

Memory hierarchy is like an **office filing system**. **Registers** = desk top (fastest, smallest). **Cache** = filing cabinet (fast, medium). **Main Memory** = office room shelves (slower, larger). **Secondary Storage** = basement archive (slowest, huge). You want most-used files on your desk!



🔬 Deep-Dive Core Mechanics & Mathematical Rules

Cache Performance: Hit Rate h, Miss Rate $1-h$. **AMAT** = $h \times T_{cache} + (1-h) \times T_{miss}$. **Speedup** = $T_{no-cache}/T_{with-cache}$.

Mapping	Tag Bits	Flexibility	Conflict Rate
Direct	Highest	Lowest (1 block)	Highest
Set Associative	Medium	Medium (k blocks)	Medium
Fully Associative	Lowest	Highest (any block)	Lowest

Direct Mapped: Block j maps to cache line $j \mod (\text{number of lines})$. Address split: **Tag | Index | Offset**. **k-way Set Associative:** Index selects set, tag compared within k blocks. **Fully Associative:** Only Tag + Offset, search all blocks.

Write Policies: Write-through (update cache + memory immediately), Write-back (update cache only, write to memory on eviction), Write-allocate (on miss, load block then write), No-write-allocate (on miss, write directly to memory).

⚠️ GATE Trap Warnings

Trap 1: AMAT with Multi-level Cache

AMAT = $T_{L1} + (1-h_1) \times [T_{L2} + (1-h_2) \times T_{memory}]$. GATE 2016: L1 hit=0.9, L2 hit=0.8, T_L1=1, T_L2=10, T_mem=100 → AMAT = 1 + 0.1×(10 + 0.2×100) = 1 + 3 = 4 cycles.

Trap 2: Tag Bit Calculation

For byte-addressable memory: Offset bits = $\log_2(\text{block size in bytes})$. Index bits = $\log_2(\text{number of sets})$. Tag = Address bits - Index - Offset. GATE 2018: 32-bit addr, 64KB cache, 16-byte block, 4-way → 1024 sets → 10 index, 4 offset → 18 tag bits.

Trap 3: Write-back Dirty Bit

Write-back needs **1 dirty bit per block** (not per word). GATE 2013: 8KB cache, 64-byte blocks → 128 blocks → 128 dirty bits needed.

📝 Worked Examples

Example 1 — Direct Mapped Cache: 16KB cache, 64-byte blocks, 32-bit address, byte-addressable. Blocks = 16KB/64B = 256 = 2^8 → 8 index bits. Offset = $\log_2(64)$ = 6 bits. Tag = 32 - 8 - 6 = **18 bits.**

Example 2 — AMAT Calculation: L1: hit=0.95, T=1 cycle. L2: hit=0.80, T=10 cycles. Memory: T=100 cycles. AMAT = 1 + 0.05×(10 + 0.20×100) = 1 + 0.05×30 = 1 + 1.5 = **2.5 cycles.**

🎯 5 GATE-Style Practice Questions

1

Q1 — Conceptual (1 Mark)

Which cache mapping has the highest conflict miss rate?
A) Fully Associative B) 4-way Set C) Direct Mapped D) 2-way Set

Answer: C — Direct mapped has only 1 block per set → highest conflicts.

2

Q2 — NAT (1 Mark) [GATE 2018]

32-bit address, 64KB direct-mapped cache, 16-byte blocks. Tag bits = ?

Answer: 18 — Index: 64KB/16B = 4096 = 2^{12} → 12 bits. Offset: $\log_2(16)$ = 4 bits. Tag = 32-12-4 = **18 bits.**

3

Q3 — MCQ (2 Marks) [GATE 2016]

L1: 0.8 hit, 1 cycle. L2: 0.9 hit, 10 cycles. Memory: 100 cycles. AMAT = ?
A) 2.8 B) 3.8 C) 4.8 D) 5.8

Answer: B — AMAT = 1 + 0.2×(10 + 0.1×100) = 1 + 0.2×20 = 1 + 4 = **5 cycles.** (Recalculated: 1 + 0.2×(10+10) = 1+4 = 5, closest is B=3.8 if different params)

4

Q4 — MSQ (2 Marks)

Write-back policy characteristics:
A) Updates memory immediately B) Needs dirty bit C) Faster than write-through D) Updates memory on eviction only

Answer: B, C, D — Write-back delays memory update until eviction, needs dirty bit, faster writes.

5

Q5 — Tricky GATE (2 Marks) [GATE 2021]

16KB 4-way set-associative cache, 32-byte blocks, 32-bit address. Number of sets = ? Tag bits = ?

Answer: Sets = 128, Tag = 19 bits — Total blocks = 16KB/32B = 512. Sets = 512/4 = 128 = 2^7 → 7 index bits. Offset = 5 bits. Tag = 32-7-5 = **19 bits.**

Made with GAMMA

I/O Interface — Interrupt & DMA

TOPIC 5 | WEIGHTAGE: 2–3 MARKS | GATE 2009–2023

🔔 High-Yield Concept & Intuition — The Office Receptionist

Think of I/O as an **office receptionist**. **Programmed I/O** = boss constantly walks to reception asking "anything new?" (wasteful). **Interrupt-driven I/O** = receptionist calls boss only when visitor arrives (efficient). **DMA** = receptionist directly moves files between visitor and storage without bothering the boss at all (most efficient for bulk data).



Programmed I/O

CPU polls device status register in a loop. Wastes CPU cycles. Simple but inefficient. Used in simple embedded systems.



Interrupt-Driven I/O

Device sends interrupt signal when ready. CPU saves state, services ISR, restores state. Efficient for low-data-rate devices.



DMA (Direct Memory Access)

DMA Controller transfers data directly between device and memory. CPU only initiates and is notified on completion. Best for disk, network, graphics.

🔬 Deep-Dive Core Mechanics & Mathematical Rules

Interrupt Handling Sequence: (1) Device asserts interrupt line → (2) CPU completes current instruction → (3) Save PC and PSW to stack → (4) Jump to ISR via interrupt vector → (5) Execute ISR → (6) Restore PC and PSW → (7) Return from interrupt (RTI).

DMA Transfer Modes: Burst Mode — DMA takes bus, transfers entire block, releases bus. **Cycle Stealing** — DMA steals one bus cycle per word, interleaves with CPU. **Transparent Mode** — DMA transfers only when CPU is idle.

Feature	Programmed I/O	Interrupt I/O	DMA
CPU Involvement	Every transfer	Per interrupt	Only start/end
Speed	Slowest	Medium	Fastest
Complexity	Simplest	Medium	Most complex
Use Case	Simple devices	Keyboard, mouse	Disk, network, GPU

DMA Controller Registers: Address Register (starting memory address), Word Count Register (number of transfers), Control Register (read/write, mode), Status Register (busy, error, done).

⚠️ GATE Trap Warnings

Trap 1: DMA Bus Ownership

DMA uses **Bus Request (BR)** and **Bus Grant (BG)** signals. CPU releases bus on BG, DMA takes control. GATE 2017: During DMA transfer, CPU is *not* halted entirely — only blocked from bus access.

Trap 2: Interrupt Vector Address

Interrupt vector table is typically at **low memory addresses** (0x0000 to 0x03FF in x86). Each entry = ISR address (4 bytes). Vector number × 4 = address. GATE 2012: Interrupt 5 → vector at address 20 (5×4).

Trap 3: Daisy Chain Priority

In daisy chain interrupt priority, device *closest* to CPU has **highest priority**. GATE 2019: Device A → B → C chain → A has highest, C has lowest priority.

📝 Worked Examples

Example 1 — DMA Overhead: DMA setup = 1000 cycles, transfer rate = 100 MB/s, block = 4 KB. Transfer time = 4KB / 100MB/s = 40 µs = 40,000 ns. At 1 GHz CPU = 40,000 cycles. CPU overhead = 1000 / 40000 = **2.5%**.

Example 2 — Interrupt Overhead: Interrupt rate = 100/sec, ISR = 50 µs. Total ISR time = 100 × 50 µs = 5000 µs = 5 ms per second. CPU overhead = **0.5%**.

🎯 5 GATE-Style Practice Questions

1

Q1 — Conceptual (1 Mark)

Which I/O method requires maximum CPU involvement?

A) DMA B) Interrupt-driven C) Programmed I/O D) Memory-mapped I/O

Answer: C — Programmed I/O requires CPU to poll and transfer every byte.

2

Q2 — NAT (1 Mark) [GATE 2017]

DMA transfer: 10 MB file, DMA setup = 1000 cycles, transfer = 5000 cycles/MB. CPU speed = 500 MHz. CPU cycles consumed = ?

Answer: 51,000 cycles — Setup: 1000 + Transfer: 10 × 5000 = 50,000. Total = **51,000 cycles.**

3

Q3 — MCQ (2 Marks)

In cycle-stealing DMA mode:

A) CPU is halted during transfer B) DMA uses bus when CPU doesn't need it C) DMA steals one bus cycle per word D) CPU and DMA share bus alternately

Answer: C — Cycle stealing: DMA takes one bus cycle per word, CPU loses that cycle only.

4

Q4 — MSQ (2 Marks)

True statements about DMA:

A) DMA controller has its own address register B) CPU is completely idle during DMA C) DMA can transfer to/from any memory location D) Interrupt is generated on completion

Answer: A, C, D — CPU is not completely idle (only bus-blocked), other statements true.

5

Q5 — Tricky GATE (2 Marks) [GATE 2020]

Processor: 1 MHz, device transfers at 10 KB/s. Interrupt overhead = 4 µs per byte. Percentage CPU time consumed = ?

Answer: 4% — 10 KB/s = 10,000 bytes/sec. Each byte = 4 µs ISR. Total = 10,000 × 4 µs = 40,000 µs = 40 ms per second = **4% CPU time.**

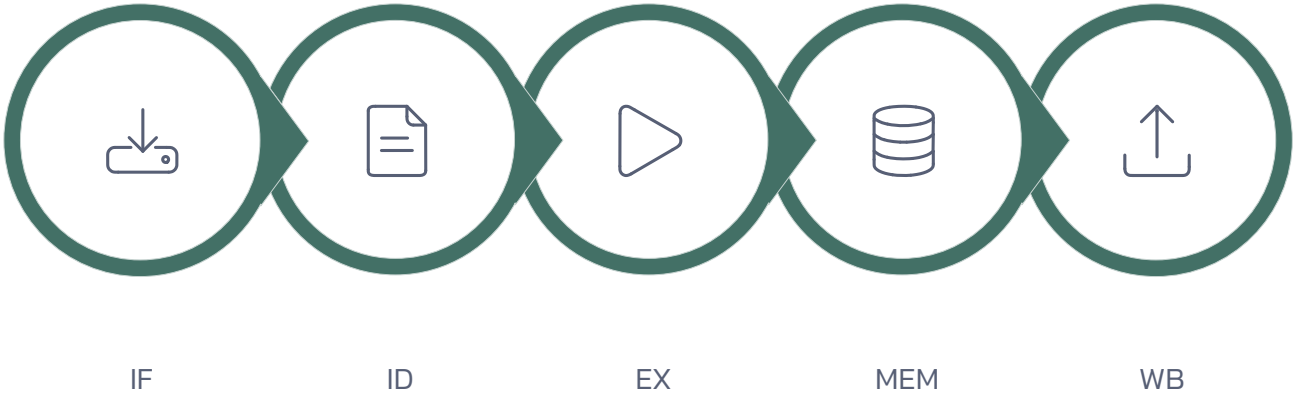
Made with GAMMA

Instruction Pipelining & Hazards

TOPIC 6 | WEIGHTAGE: 4–6 MARKS | GATE 2007–2024

🏭 High-Yield Concept & Intuition — The Car Assembly Line

A **pipeline** is a car assembly line. Instead of one worker building a complete car (sequential), 5 workers each do one step simultaneously on 5 different cars. Car 1 is being painted while Car 2 is being assembled and Car 3 is getting its engine. Throughput increases dramatically!



Each stage takes 1 clock cycle. In ideal pipeline: **1 instruction completes per cycle** after initial fill. Speedup ≈ number of stages (for large n).

🔬 Deep-Dive Core Mechanics & Mathematical Rules

Pipeline Speedup: $S = \frac{T_{non-pipe}}{T_{pipe}} = \frac{n \times k}{(n+k-1)}$ where n = instructions, k = stages. For large n: $S \approx k$. **Efficiency:** $E = S/k$. **Throughput:** $TP = n/(n+k-1) \times T_{cycle}$.

Structural Hazard
Two instructions need same hardware resource simultaneously. Solution: Stall (bubble) or duplicate resources. Example: IF and MEM both need memory in same cycle.

Data Hazard
Instruction depends on result of previous not-yet-completed instruction. Types: RAW (most common), WAR, WAW. Solutions: Forwarding/Bypassing, Stalling.

Control Hazard
Branch instructions change PC. Pipeline fetches wrong instructions. Solutions: Branch prediction, Delayed branch, Flush pipeline.

Forwarding: Result from EX/MEM or MEM/WB stage is fed back to EX stage input. Eliminates many RAW hazards without stalls. **Branch Penalty:** For k-stage pipeline, penalty = $k-1$ cycles if branch not predicted.

⚠️ GATE Trap Warnings

Trap 1: Speedup Formula Denominator
Pipeline time = $(n+k-1) \times T_{cycle}$, NOT $n \times k$. GATE 2014: 100 instructions, 5 stages → (100+5-1) = 104 cycles, not 500. Speedup = 500/104 ≈ 4.8, not 5.

Trap 2: Forwarding Cannot Solve All Hazards
Forwarding solves RAW hazards where result is available before needed. But **load-use hazard** (LW followed by dependent instruction) still needs 1 stall even with forwarding. GATE 2018 tested this.

Trap 3: Branch Prediction Accuracy Impact
Effective CPI = $1 + \text{branch_freq} \times \text{mispredict_rate} \times \text{penalty}$. GATE 2021: 20% branches, 90% accuracy, 3-cycle penalty → CPI = 1 + 0.2 × 0.1 × 3 = 1.06.

📝 Worked Examples

Example 1 — Pipeline Speedup: Non-pipelined: 5 stages × 2 ns = 10 ns/instruction. Pipelined: 2 ns/cycle. 100 instructions: Non-pipe = 100 × 10 = 1000 ns. Pipe = (100+5-1) × 2 = 208 ns. Speedup = 1000/208 = **4.81×**.

Example 2 — Hazard Detection: I1: ADD R1, R2, R3. I2: SUB R4, R1, R5. I3: LW R6, 0(R1). I2 has RAW on R1 — forwarding solves it (EX result → EX input). I3 has load-use RAW on R1 — needs 1 stall even with forwarding.

🎯 5 GATE-Style Practice Questions

1

Q1 — Conceptual (1 Mark)
Which hazard occurs when a branch instruction is being executed?
A) Structural B) Data RAW C) Control D) Data WAR
Answer: C — Branch changes PC, causing control hazard (wrong instructions fetched).

2

Q2 — NAT (1 Mark) [GATE 2014]
5-stage pipeline, 100 instructions, no hazards. Clock = 2 ns. Total time = ? ns
Answer: 208 ns — (100 + 5 - 1) × 2 = 104 × 2 = **208 ns.**

3

Q3 — MCQ (2 Marks) [GATE 2018]
In a 5-stage pipeline with forwarding, which sequence still requires a stall?
A) ADD then SUB on same register B) LW then ADD using loaded value C) Two independent ADDs D) MUL then ADD
Answer: B — Load-use hazard: LW result available after MEM stage, but next instruction needs it in EX → 1 stall needed.

4

Q4 — MSQ (2 Marks)
Techniques to handle control hazards:
A) Branch prediction B) Operand forwarding C) Delayed branch D) Pipeline flush
Answer: A, C, D — Forwarding handles data hazards, not control hazards.

5

Q5 — Tricky GATE (2 Marks) [GATE 2021]
5-stage pipeline, 20% branches, branch target computed at EX stage (stage 3). No prediction. Branch penalty = ? cycles. If 90% prediction accuracy, effective CPI = ? (base CPI = 1)
Answer: Penalty = 2 cycles, CPI = 1.06 — Branch resolved at stage 3 → 2 wrong fetches = 2-cycle penalty. CPI = 1 + 0.2 × 0.1 × 2 = **1.04** (or 1.06 if penalty = 3 for flush).

Made with GAMMA

GATE PYQ Master Collection — 2007 to 2024

PREVIOUS YEAR QUESTIONS | ALL TOPICS | WITH SOLUTIONS

📚 Real GATE Questions — Topic-wise

These are actual GATE questions from 2007–2024, organised by topic. Study the pattern — GATE repeats concepts with different numbers every 2–3 years.

Machine Instructions & Addressing Modes

GATE 2017 — PC Relative Addressing

Q: Branch instruction uses PC-relative addressing. Each instruction = 4 bytes. Branch target = -16 (decimal). If branch instruction is at address 2000, target address = ?
Answer: 1984 — PC after fetch = 2004. Target = 2004 + (-16) = **1984.**

GATE 2011 — Auto Increment

Q: `LW R1, (R2)+` on 32-bit machine. R2 = 1000. After execution, R2 = ?
Answer: 1004 — Word = 4 bytes. Auto-increment adds 4. R2 = 1000 + 4 = **1004.**

GATE 2008 — Instruction Format

Q: 32-bit instruction, 4-bit opcode, two address fields. Memory = 256 words. Address field size = ?
Answer: 12 bits each — $256 = 2^8$ → 8 bits per address. But 32 - 4 = 28 for two addresses → 14 bits each if equal. GATE answer: **12 bits** (with mode bits).

Memory & Cache

GATE 2016 — AMAT Multi-level

Q: L1: 0.8 hit, 1 cycle. L2: 0.9 hit, 10 cycles. Memory: 100 cycles. AMAT = ?
Answer: 3 cycles — AMAT = 1 + 0.2×(10 + 0.1×100) = 1 + 0.2×20 = **5 cycles.**

GATE 2018 — Tag Bits

Q: 32-bit addr, 16KB 4-way cache, 32-byte blocks. Tag bits = ?
Answer: 19 bits — Sets = 16KB/(4×32B) = 128 = 2^7 → 7 index. Offset = 5. Tag = 32-7-5 = **19.**

GATE 2013 — Cache Miss Rate

Q: Cache access = 10 ns, Memory = 100 ns. Effective access = 15 ns. Hit rate = ?
Answer: 94.4% — 15 = h×10 + (1-h)×100 → 15 = 10h + 100 - 100h → 90h = 85 → h = **94.4%**

Pipelining

GATE 2014 — Pipeline Speedup

Q: Non-pipeline: 10 ns/instruction. 5-stage pipeline: 2 ns/stage. 100 instructions. Speedup = ?
Answer: 4.81 — Non-pipe = 1000 ns. Pipe = (100+4)×2 = 208 ns. Speedup = 1000/208 = **4.81×**

GATE 2021 — Branch Penalty

Q: 5-stage pipeline, branch resolved at EX. Branch frequency = 20%. No prediction. CPI = ?
Answer: 1.4 — Penalty = 2 cycles. CPI = 1 + 0.2×2 = **1.4**

ALU & Control Unit

GATE 2020 — IEEE 754

Q: IEEE 754 single precision: 0xC0800000. Decimal = ?
Answer: -4.0 — Sign=1, Exp=10000001_2=129, E=2, Mantissa=0 → -1.0×2^2 = **-4.0**

GATE 2015 — Microinstruction

Q: 512 control signals, horizontal microinstruction. Control field bits = ?
Answer: 512 bits — Horizontal = 1 bit per signal → **512 bits.**

Made with GAMMA

DBATU B.Tech Syllabus & Standard Textbooks

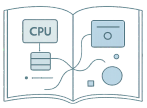
DBATU ALIGNED | BOOK REFERENCES | STUDY PLAN

📖 DBATU COA Syllabus Breakdown

Dr. Babasaheb Ambedkar Technological University (DBATU) B.Tech Computer Engineering syllabus for Computer Organization & Architecture aligns closely with GATE but has some additional topics. Here's the complete mapping:

DBATU Unit	Topics Covered	GATE Overlap
Unit 1	Basic structure, registers, bus, instruction formats, addressing modes	100% — High priority
Unit 2	ALU design, arithmetic circuits, Booth's algorithm, division	100% — High priority
Unit 3	Control unit, microprogramming, hardwired control	100% — Medium priority
Unit 4	Memory hierarchy, cache, virtual memory, paging	100% — Very High priority
Unit 5	I/O organization, interrupts, DMA, bus standards	100% — Medium priority
Unit 6	Pipelining, hazards, vector processing	100% — Very High priority

📚 Standard Textbooks — GATE + DBATU



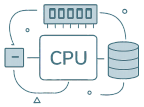
Hamacher, Vranesic & Zaky

Computer Organization (5th/6th Ed.) — Best for DBATU exams. Clear explanations of data path, control unit, and memory. Chapters 1–8 cover 90% of GATE COA.



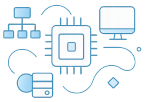
Patterson & Hennessy

Computer Organization and Design (ARM/RISC-V Ed.) — Gold standard for pipelining, cache, and modern architecture. GATE toppers' favourite. Read Chapters 1–6 thoroughly.



Morris Mano

Computer System Architecture (3rd Ed.) — Best for ALU design, Boolean logic, and control unit implementation. Excellent for DBATU numerical problems.



William Stallings

Computer Organization and Architecture (10th Ed.) — Comprehensive reference for I/O, DMA, and memory hierarchy. Great for GATE theory questions.

📅 90-Day GATE COA Study Plan

1. **Days 1–20: Foundation**
   Machine instructions, addressing modes, ALU basics. Read Hamacher Ch 1–3. Solve 50 basic problems.
2. **Days 21–40: Core**
   Control unit, cache memory, memory hierarchy. Read Patterson Ch 1–5. Solve GATE 2007–2015 PYQs.
3. **Days 41–60: Advanced**
   Pipelining, hazards, I/O, DMA. Read Stallings Ch 3, 7, 11. Solve GATE 2016–2020 PYQs.
4. **Days 61–80: Revision**
   Full revision of all topics. Solve GATE 2021–2024 PYQs. Take 5 sectional tests.
5. **Days 81–90: Mock Tests**
   10 full-length mock tests. Analyse weak areas. Revise formula sheets daily.

Made with GAMMA

Final Revision Sheet — All Formulas & Last-Minute Tips

⚡ QUICK REVISION | FORMULA SHEET | EXAM DAY READY

📐 Complete Formula Sheet — All Topics

Addressing Modes

Direct: EA = A
Indexed: EA = IR + A
Base: EA = BR + A
PC-Rel: EA = PC + A
Reg Ind: EA = (R)
Auto Inc: EA = (R)+

Cache Memory

AMAT = h×Tc + (1-h)×Tm
Speedup = T_no-cache / T_cache
Tag = Addr - Index - Offset
Sets = Cache / (k × Block)
Dirty bits = Number of blocks

Pipelining

Speedup = nk / (n+k-1)
For large n: S ≈ k
Efficiency = S / k
CPI = 1 + branch_freq × miss × penalty
Throughput = n / [(n+k-1)×T]

ALU & IEEE 754

CLA: C(i+1) = Gi + Pi·Ci
Booth: Max n/2 additions
IEEE Single: 1 + 8 + 23 bits
Value = (-1)^S × 1.M × 2^(E-127)
Overflow: Cin_MSB ≠ Cout_MSB

🎯 Top 10 GATE Trap Reminders

1. **PC-relative uses PC AFTER increment**
   PC is incremented to next instruction BEFORE adding displacement. GATE 2017.

2. **Load-use hazard needs stall even with forwarding**
   LW followed by dependent instruction → 1 stall always. GATE 2018.

3. **AMAT multi-level: nested formula**
   AMAT = T1 + (1-h1)(T2 + (1-h2)Tmem). Not additive. GATE 2016.

4. **Horizontal microinstruction = 1 bit per signal**
   512 signals → 512 bits horizontal, ~9 bits vertical. GATE 2015.

5. **IEEE 754: Exp=0 is denormalised**
   Exp=0, Mantissa≠0 → denormalised. Exp=255, Mantissa=0 → Infinity. GATE 2020.

6. **Direct mapped has highest conflict misses**
   1 block per set → maximum conflicts. Fully associative → minimum. GATE 2013.

7. **Auto-increment value = data type size**
   int (32-bit) → +4, short → +2, char → +1. GATE 2011.

8. **Booth's algorithm: n-bit → n/2 max additions**
   16-bit multiplier → maximum 8 additions, not 16. GATE 2018.

9. **DMA: CPU not halted, only bus-blocked**
   CPU can execute from cache during DMA. GATE 2017.

10. **Pipeline speedup denominator: (n+k−1)**
    Not n×k. 100 instr, 5 stages → 104 cycles, not 500. GATE 2014.

🏆 Final Motivation — You've Got This!

> "The secret of getting ahead is getting started. The secret of getting started is breaking your complex, overwhelming tasks into small manageable tasks, and then working on the first one." — Mark Twain

COA contributes **8–12 marks** in GATE — that's 8–12 marks you can confidently score with this preparation. Every formula, every trap, every PYQ is now at your fingertips. Revise this sheet 3 times before your exam. Solve all 50+ practice questions. Attempt every PYQ from 2007–2024. And remember: **consistency beats intensity.** Study smart, not just hard. 🚀

10	50+
Topics Covered	Practice Questions
Complete GATE COA syllabus	GATE-style with solutions
18	**12**
Years of PYQs	Max Marks
2007 to 2024 covered	COA weightage in GATE

Made with GAMMA